



**IPN**

**INSTITUTO POLITECNICO NACIONAL**

**ESCUELA SUPERIOR DE INGENIERÍA MECANICA Y ELECTRICA**

**ESIME**

**MATERIA: SISTEMAS DE INFORMACION WEB**

**PROFESOR: ENTZANA GARDUÑO YVENTZ**

**EQUIPO:**

**GUSTAVO AGUILAR FRANCO**

**ANGEL SAMUEL ALVARADO TRONCOSO**

**NAVARRETE HERNÁNDEZ ROSALÍA**

**GRUPO: 4EV24**





## ETAPA 1

### **Problemática y propósito:**

Actualmente muchas bibliotecas pequeñas llevan el control de sus libros mediante registros físicos o archivos dispersos, lo que puede ocasionar problemas como pérdida de información, errores en los préstamos o dificultad para localizar libros disponibles.

La problemática principal consiste en la falta de organización y control eficiente de los recursos de la biblioteca.

Por esta razón se desarrolló la base de datos \*Universo Literario\*, cuyo propósito es centralizar toda la información en un solo sistema. Esto permite administrar usuarios, registrar préstamos, controlar devoluciones y generar un historial de movimientos de manera rápida y segura.

De esta forma se mejora la gestión de la biblioteca y se brinda un mejor servicio a los usuarios.

### **¿Qué es MySQL?**

MySQL es uno de los sistemas gestores de bases de datos más utilizados en el mundo debido a su facilidad de uso, velocidad y confiabilidad.

Su nombre está compuesto por dos partes:

\* \*My\*, que corresponde al nombre de la hija de uno de sus creadores.

\* \*SQL\*, que significa \*Structured Query Language o Lenguaje Estructurado de Consultas.

MySQL trabaja bajo el modelo relacional, lo que significa que la información se almacena en tablas relacionadas entre sí. Gracias a esto es posible organizar grandes cantidades de datos y realizar búsquedas o consultas de forma eficiente.

Actualmente es utilizado por empresas, universidades, hospitales y sitios web para administrar información crítica.

### **¿Para qué sirve MySQL?**

MySQL tiene aplicaciones en una gran variedad de áreas debido a su capacidad para almacenar y administrar información.

Algunos ejemplos son:



- \* Control de inventarios en empresas.
- \* Sistemas de ventas y facturación.
- \* Administración de bibliotecas.
- \* Sistemas escolares.
- \* Plataformas de comercio electrónico.
- \* Sitios web con usuarios registrados.
- \* Gestión de préstamos y devoluciones.

En este proyecto se utiliza para controlar toda la información de una biblioteca, permitiendo que cada operación quede registrada dentro de la base de datos.

Esto facilita el acceso a la información y mejora la toma de decisiones.

### **Requerimientos de instalación para el software Mysql.**

| <b>Componente</b>        | <b>Requerimiento Mínimo</b>                |
|--------------------------|--------------------------------------------|
| Sistema Operativo        | Windows 10/11, Linux o macOS               |
| Procesador               | 1 GHz o superior                           |
| Memoria RAM              | 2 GB (4 GB recomendados)                   |
| Espacio en Disco         | 500 MB libres (1 GB recomendado)           |
| Resolución de Pantalla   | 1024 × 768                                 |
| Conexión a Internet      | Requerida para descarga e instalación web  |
| Privilegios de Usuario   | Permisos de Administrador                  |
| Puerto de Comunicación   | 3306 (predeterminado)                      |
| Framework .NET (Windows) | Versión 4.5 o superior                     |
| Navegador Web            | Recomendado para descargas y documentación |

### **Instalación de Mysql**

Antes de poder trabajar con una base de datos es necesario instalar el gestor MySQL en el equipo. Durante la instalación se descargan y configuran los componentes necesarios para que el servidor de base de datos funcione correctamente.



- Paso 1 :

MySQL Community Downloads

Login Now or Sign Up for a free account.

An Oracle Web Account provides you with the following advantages:

- Fast access to MySQL software downloads
- Download technical White Papers and Presentations
- Post messages in the MySQL Discussion Forums
- Report and track bugs in the MySQL bug system

[Login »](#)  
using my Oracle Web account

[Sign Up »](#)  
for an Oracle Web account

MySQL.com is using Oracle SSO for authentication. If you already have an Oracle Web account, click the Login link. Otherwise, you can sign up for a free account by clicking the Sign Up link and following the instructions.

[No thanks, just start my download.](#)

ORACLE © 2025 Oracle

[Privacy / Do Not Sell My Info](#) | [Terms of Use](#) | [Trademark Policy](#) | [Cookie Preferences](#)

Recent download history

| File Name              | Size   | Status |
|------------------------|--------|--------|
| mysql-9.5.0-winx64.msi | 171 MB | Done   |

[Full download history](#)

Descargar el instalador oficial desde la página de MySQL.

- Paso2:



Ejecutar el programa de instalación.

- Paso 3:

The screenshot shows the Microsoft Learn page for Visual Studio 2022. The left sidebar contains a navigation menu with options like 'Modules', 'Redistribute Visual C++ ActiveX Controls', 'Redistribute the MFC Library', 'Redistribute an ATL application', 'Latest Supported Visual C++ Redistributable Downloads', 'How to audit Visual C++ Runtime version usage', 'Deployment examples', 'Redistribute web client applications', 'ClickOnce deployment for Microsoft C++ applications', 'Run a C++ -clr application on a previous runtime version', 'C++ Attributes for COM and .NET', 'Attribute programming FAQ', 'Attributes by group', 'Attributes by usage', and 'Attributes alphabetical reference'. The main content area has a 'Note' box stating: 'Are you here to download Visual Studio? Go to [Visual Studio downloads](#).' Below this, a paragraph explains that the topic is for developers who need to install the Visual C++ runtime libraries with their app. A table titled 'Expand table' lists the latest supported redistributable versions for Visual Studio 2017-2026, 2015, 2013, 2012, 2010, 2008, and 2005. A footer note states: 'Redistribution is permitted only for licensed Visual Studio users, as described in the [Visual Studio license terms](#). For details on redistributing Visual C++ files, see [Redistributing Visual C++ Files](#).'

| Version                                            | Section                                        |
|----------------------------------------------------|------------------------------------------------|
| Latest supported v14 (for Visual Studio 2017–2026) | Latest supported Redistributable version       |
| Visual Studio 2015                                 | <a href="#">Visual Studio 2015 (VC++ 14.0)</a> |
| Visual Studio 2013                                 | <a href="#">Visual Studio 2013 (VC++ 12.0)</a> |
| Visual Studio 2012                                 | <a href="#">Visual Studio 2012 (VC++ 11.0)</a> |
| Visual Studio 2010                                 | <a href="#">Visual Studio 2010 (VC++ 10.0)</a> |
| Visual Studio 2008                                 | <a href="#">Visual Studio 2008 (VC++ 9.0)</a>  |
| Visual Studio 2005                                 | <a href="#">Visual Studio 2005 (VC++ 8.0)</a>  |

Seleccionar el tipo de instalación de acuerdo con las necesidades del proyecto.

- Paso 4:

The screenshot shows the Microsoft Learn page for Visual Studio 2022, specifically the 'Latest supported redistributable version' section. The left sidebar is the same as in the previous screenshot. The main content area has a paragraph explaining how to find the version number of the latest redistributable. A table titled 'Expand table' lists the latest supported redistributable versions for ARM64, x86, and x64 architectures. A footer note states: 'Download other versions, including long term servicing release channel (LTSC) versions, from [my.visualstudio.com](#).' Below this, a 'Notes' section mentions that some downloads are currently available on [my.visualstudio.com](#) and that users should log in using a Visual Studio Subscription account or create a free account by choosing the 'No account? Create one!' link.

| Architecture | Link                                                                                          | Notes                                                                                                                                                                                                                                                    |
|--------------|-----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ARM64        | <a href="https://aka.ms/vc14/vc_redist.arm64.exe">https://aka.ms/vc14/vc_redist.arm64.exe</a> | Permalink for latest supported ARM64 version                                                                                                                                                                                                             |
| x86          | <a href="https://aka.ms/vc14/vc_redist.x86.exe">https://aka.ms/vc14/vc_redist.x86.exe</a>     | Permalink for latest supported x86 version                                                                                                                                                                                                               |
| x64          | <a href="https://aka.ms/vc14/vc_redist.x64.exe">https://aka.ms/vc14/vc_redist.x64.exe</a>     | Permalink for latest supported x64 version. The x64 Redistributable package contains both ARM64 and x64 binaries. This package makes it easy to install required Visual C++ ARM64 binaries when the x64 Redistributable is installed on an ARM64 device. |

Verificar que todos los componentes requeridos estén seleccionados.



- Paso 5:

Version: Visual Studio 2022

Find by title

Modules

- Redistribute Visual C++ ActiveX Controls
- Redistribute the MFC Library
- Redistribute an ATL application
- Latest Supported Visual C++ Redistributable Downloads
- How to audit Visual C++ Runtime version usage

> Deployment examples

- Redistribute web client applications
- ClickOnce deployment for Microsoft C++ applications
- Run a C++ -clr application on a previous runtime version

C++ Attributes for COM and .NET

Attribute programming FAQ

> Attributes by group

> Attributes by usage

> Attributes alphabetical reference

frequently. To find the version number of the latest redistributable, download the one you're interested in using one of the following links. Then, look at its properties using Windows File Explorer. In the **Details** pane, the **File version** contains the version of the redistributable.

### Latest supported redistributable version

| Architecture | Link                                                                                                                                |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------|
| ARM64        | <a href="#">http://go.microsoft.com/fwlink/?LinkId=321823&amp;clcid=0x409&amp;lang=en-us&amp;arch=arm64&amp;version=14.50.35719</a> |
| X86          | <a href="#">http://go.microsoft.com/fwlink/?LinkId=321823&amp;clcid=0x409&amp;lang=en-us&amp;arch=x86&amp;version=14.50.35719</a>   |
| X64          | <a href="#">http://go.microsoft.com/fwlink/?LinkId=321823&amp;clcid=0x409&amp;lang=en-us&amp;arch=x64&amp;version=14.50.35719</a>   |

Download other versions, including long term servicing release channel (LTSR) versions, from [my.visualstudio.com](#).

### Notes

- Some of the downloads that are mentioned in this article are currently available on [my.visualstudio.com](#). Log in using a Visual Studio Subscription account so that you can access the download links. If you're asked for credentials, use your existing Visual Studio

In this article

#### Visual C++ Redistributable v14

Unsupported legacy versions  
See also

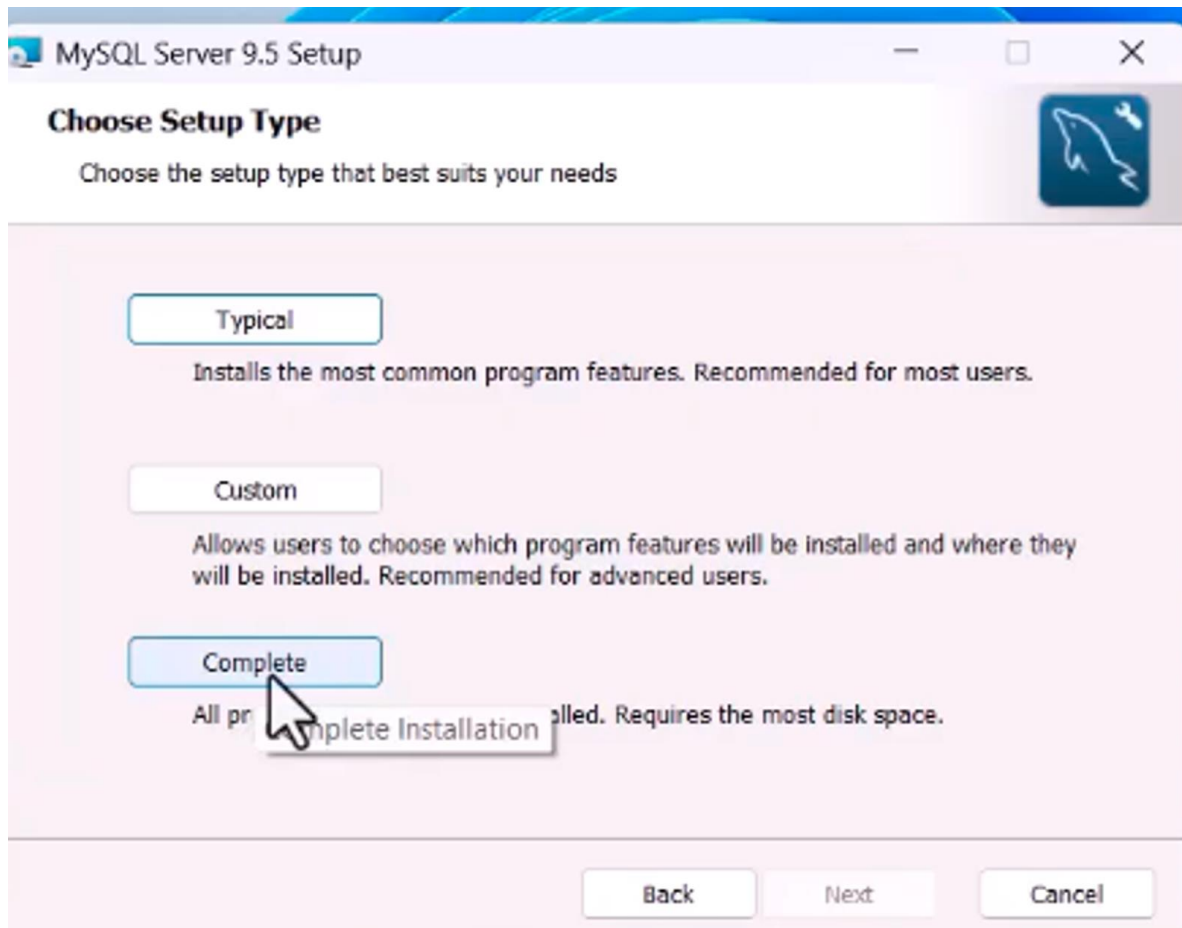
Was this page helpful?

Yes No

Realizar la instalación del servidor y las herramientas administrativas.



- Paso 6:



Esperar a que finalice el proceso y verificar que no existan errores.

Una instalación adecuada es fundamental para garantizar la estabilidad, seguridad y correcto funcionamiento de la base de datos.

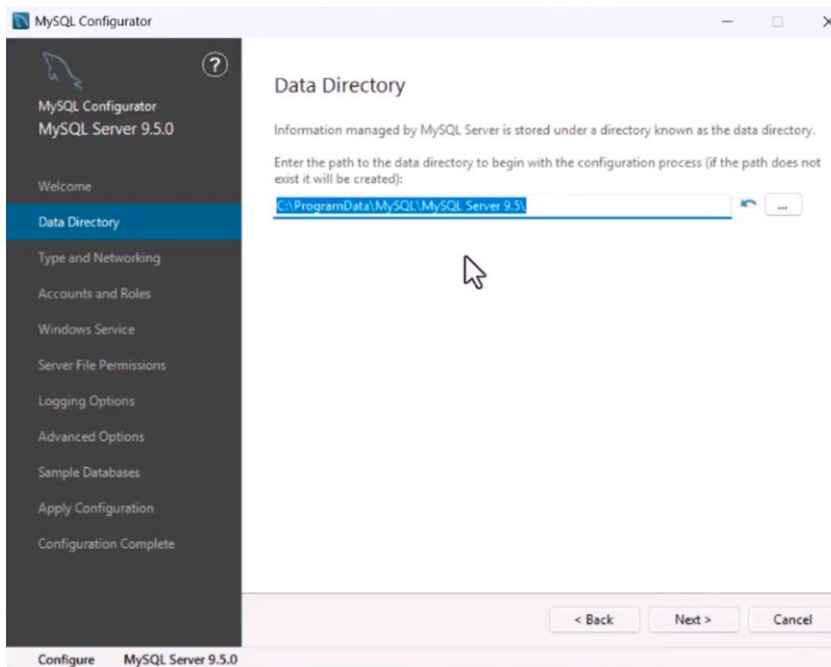
### Configuración de MySQL

Después de la instalación se realiza la configuración inicial del servidor MySQL.

Esta etapa es importante porque permite definir cómo se comunicará la base de datos con los usuarios y aplicaciones.

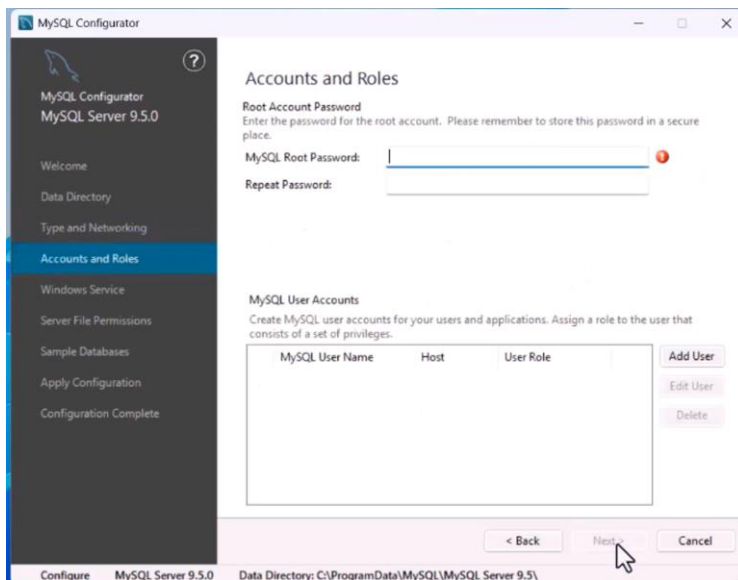


- Paso 1:



Seleccionar el modo de funcionamiento del servidor.

- Paso 2:



Configurar el puerto de conexión, normalmente el puerto 3306.





- Paso 3:

MySQL Configurator

MySQL Server 9.5.0

Welcome

Data Directory

Type and Networking

**Accounts and Roles**

Windows Service

Server File Permissions

Sample Databases

Apply Configuration

Configuration Complete

### Accounts and Roles

**Root Account Password**  
Enter the password for the root account. Please remember to store this password in a secure place.

MySQL Root Password:

Repeat Password:

Password strength: **Weak**

### MySQL User Accounts

Create MySQL user accounts for your users and applications. Assign a role to the user that consists of a set of privileges.

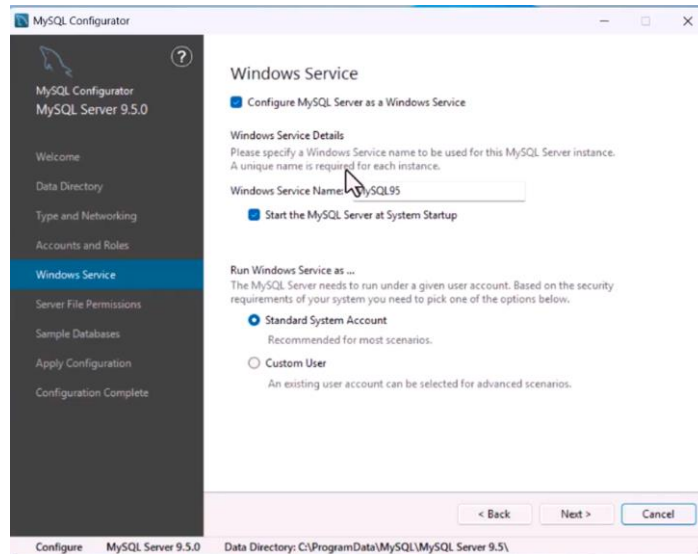
| MySQL User Name | Host | User Role |
|-----------------|------|-----------|
|-----------------|------|-----------|

< Back   Next >   Cancel

Configure   MySQL Server 9.5.0   Data Directory: C:\ProgramData\MySQL\MySQL Server 9.5\

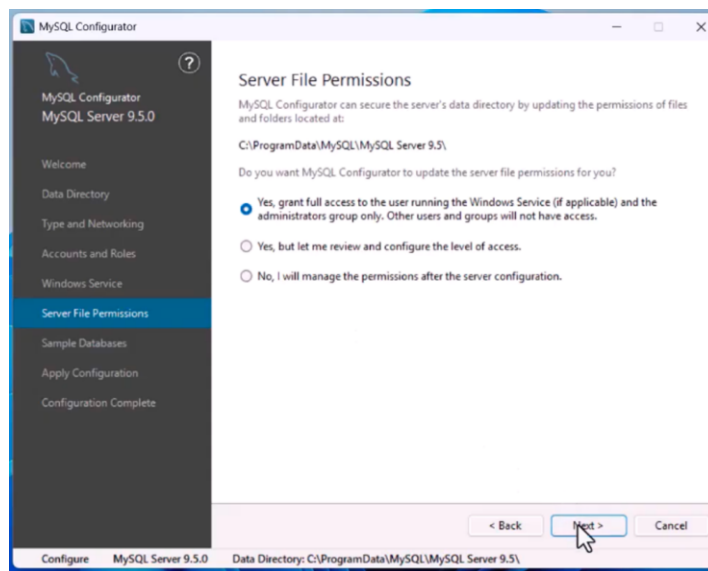
Definir las opciones de acceso a la red.

- Paso 4:



Crear la contraseña del usuario administrador o root.

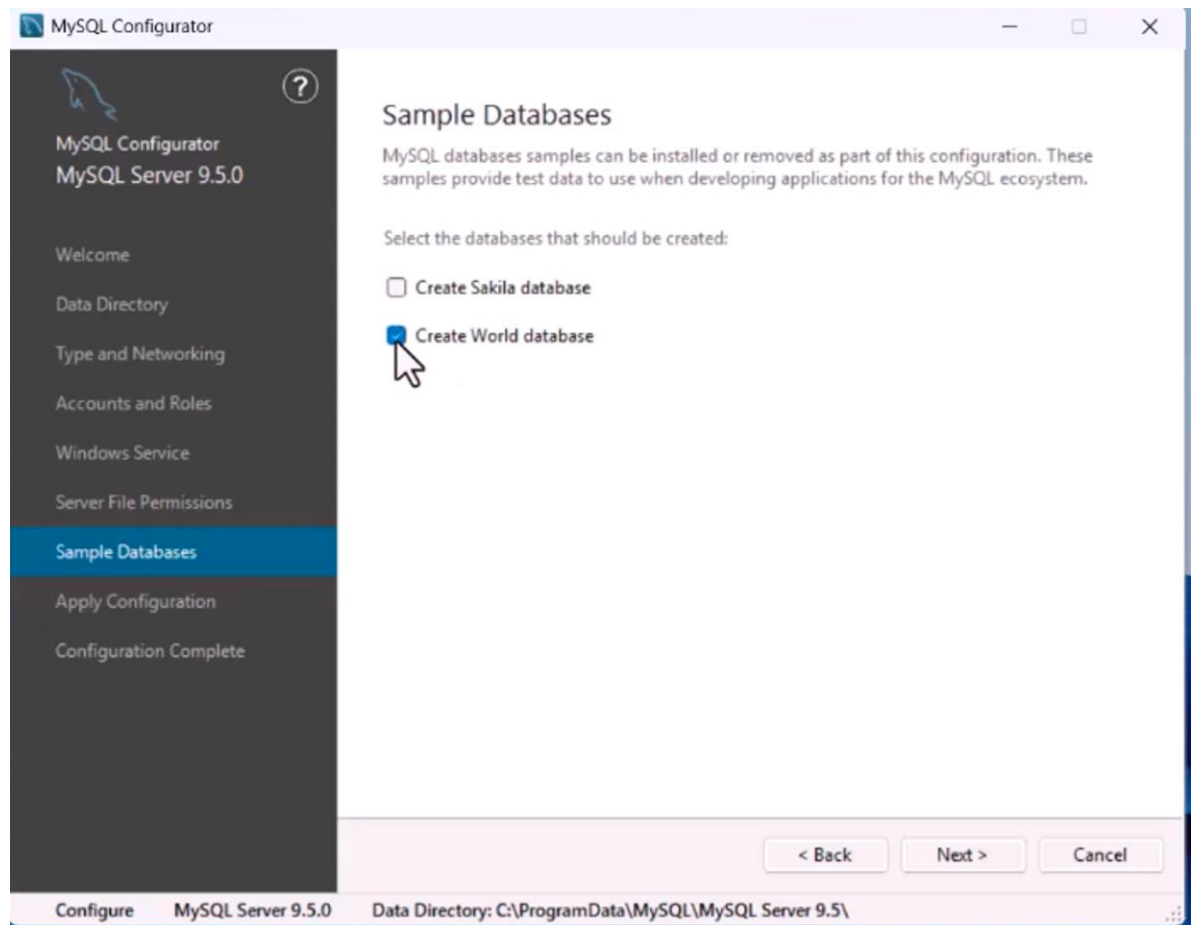
- Paso 5:



Agregar usuarios adicionales si es necesario.

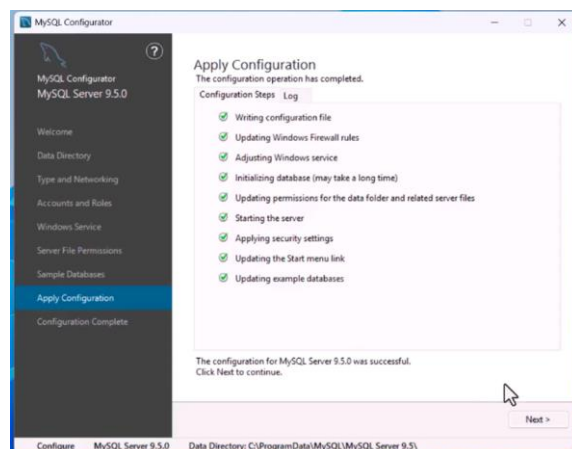


- Paso 6:



Configurar el inicio automático del servicio.

- Paso 7:





Aplicar la configuración y verificar la conexión.

La configuración adecuada permite proteger la información almacenada y garantizar que únicamente los usuarios autorizados puedan acceder a ella.

## **DDL (Data Definition Language)**

Que en español significa Lenguaje de Definición de Datos. Es una parte de SQL que se utiliza para crear y modificar la estructura de una base de datos, pero no para manipular los datos almacenados.

En tu ejemplo se está creando una base de datos llamada UniversoLiterario y varias tablas como Autores, Editoriales y Categorías.

¿Qué hace DDL?

DDL permite:

- Crear bases de datos.
- Crear tablas.
- Modificar tablas existentes.
- Eliminar tablas.
- Eliminar bases de datos.
- Cambiar nombres de objetos.

Piensa en DDL como el conjunto de herramientas para construir el "esqueleto" de una base de datos.

## **Comandos más importantes de DDL**

### **1. CREATE**

Se utiliza para crear objetos nuevos.

#### **Crear una base de datos**

```
CREATE DATABASE UniversoLiterario;
```

Resultado:



UniversoLiterario

Ahora existe una base de datos vacía.

### Seleccionar la base de datos

USE Universo Literario;

Indica que los siguientes comandos se ejecutarán dentro de esa base de datos.

### Crear una tabla

```
CREATE TABLE Autores(  
  id autor INT PRIMARY KEY,  
  nombre VARCHAR(50),  
  apellido VARCHAR(50),  
  nacionalidad VARCHAR(50)  
);
```

Aquí se crea una tabla llamada **Autores** con cuatro columnas.

| Campo        | Tipo        |
|--------------|-------------|
| id_autor     | INT         |
| nombre       | VARCHAR(50) |
| apellido     | VARCHAR(50) |
| nacionalidad | VARCHAR(50) |

## 2. ALTER

Permite modificar una tabla ya creada.

### Agregar una columna

```
ALTER TABLE Autores  
ADD fecha_nacimiento DATE;
```

Antes:

| id\_autor | nombre | apellido | nacionalidad |

Después:

| id\_autor | nombre | apellido | nacionalidad | fecha\_nacimiento |



### **Modificar una columna**

```
ALTER                                TABLE
MODIFY nombre VARCHAR(100);
```

Autores

Ahora el nombre podrá almacenar hasta 100 caracteres.

### **Cambiar nombre de una columna**

```
ALTER                                TABLE
RENAME COLUMN nombre TO nombres;
```

Autores

## **3. DROP**

Elimina completamente un objeto.

### **Eliminar una tabla**

```
DROP TABLE Autores;
```

La tabla desaparece junto con todos sus registros.

Es una acción irreversible si no tienes respaldo.

### **Eliminar una base de datos**

```
DROP DATABASE UniversoLiterario;
```

Se elimina toda la base de datos.

## **4. TRUNCATE**

Borra todos los registros de una tabla pero conserva su estructura.

Supongamos esta tabla:

### **id nombre**

1 Gabriel

2 Isabel

3 Julio

Ejecutamos:

```
TRUNCATE TABLE Autores;
```



Resultado:

**id nombre**

La tabla sigue existiendo, pero vacía.

## 5. RENAME

Cambia el nombre de un objeto.

RENAME TABLE Autores TO Escritores;

Ahora la tabla se llama:

Escritores

## Tipos de datos que aparecen en tu imagen

### INT

Almacena números enteros.

id\_autor INT

Ejemplos:

1

25

300

1000

### VARCHAR(n)

Almacena texto de longitud variable.

nombre VARCHAR(50)

Ejemplos:

Gabriel

Miguel

J. K. Rowling

de

Cervantes

### DATE

Almacena fechas.

fecha\_nacimiento DATE

Ejemplo:



1985-06-20

## Restricciones importantes

### PRIMARY KEY

Identifica de forma única cada registro.

id\_autor INT PRIMARY KEY

No puede repetirse.

Correcto:

**id\_autor**

1

2

3

Incorrecto:

**id\_autor**

1

1

### NOT NULL

Obliga a que un campo tenga valor.

nombre VARCHAR(50) NOT NULL

No se permite:

NULL

### UNIQUE

Evita valores repetidos.

correo VARCHAR(100) UNIQUE

## Relación con otros lenguajes SQL

SQL se divide en varias categorías:





### Categoría Función

|     |                        |
|-----|------------------------|
| DDL | Define estructuras     |
| DML | Manipula datos         |
| DQL | Consulta datos         |
| DCL | Controla permisos      |
| TCL | Controla transacciones |

### DML (Data Manipulation Language)

Sirve para trabajar con los datos.

INSERT  
UPDATE  
DELETE

### DQL (Data Query Language)

Sirve para consultar información.

SELECT

Por ejemplo:

SELECT \* FROM Autores;

En el sistema de biblioteca que aparece en tu imagen, el DDL fue utilizado para crear toda la estructura inicial: la base de datos **UniversoLiterario**, las tablas de **Autores**, **Editoriales**, **Categorías**, **Libros**, **Usuarios**, **Préstamos** y **Multas**. Una vez creada esa estructura, se utilizan comandos DML como INSERT, UPDATE y DELETE para almacenar y modificar la información de los libros y usuarios.

### DML (Data Manipulation Language)

**Lenguaje de Manipulación de Datos**, que es una de las categorías más importantes de SQL.

#### ¿Qué es DML?

DML es el conjunto de comandos SQL que permite **trabajar directamente con los datos almacenados dentro de las tablas**, es decir, agregar, consultar, modificar o eliminar registros.

Mientras que otros lenguajes de SQL crean la estructura de la base de datos (tablas, columnas, relaciones), DML se encarga de manipular la información que contienen esas estructuras.



Por ejemplo, en tu proyecto **Universo Literario**, cuando agregas un autor, registras un libro o eliminas una multa, estás utilizando comandos DML.

## Comandos principales de DML

### 1. INSERT

Se utiliza para agregar nuevos registros a una tabla.

Ejemplo de tu diapositiva:

```
INSERT INTO Autores VALUES  
(1,'Gabriel','García Márquez','Colombia');
```

¿Qué hace?

- Inserta un nuevo autor.
- Guarda su ID, nombre, apellido y país.

Resultado:

| id_auto<br>r | nombr<br>e | apellido       | país         |
|--------------|------------|----------------|--------------|
| 1            | Gabriel    | García Márquez | Colombi<br>a |

### 2. SELECT

Se utiliza para consultar información almacenada.

Ejemplo:

```
SELECT * FROM Autores;
```

¿Qué hace?

- Muestra todos los registros de la tabla Autores.

También se puede consultar información específica:

```
SELECT nombre, pais  
FROM Autores;
```

### 3. UPDATE

Permite modificar registros existentes.

Ejemplo:



UPDATE  
SET  
WHERE

Autores  
pais='México'  
id\_autor=1;

¿Qué hace?

- Busca al autor con ID 1.
- Cambia el valor de su país a México.

#### 4. DELETE

Permite eliminar registros.

Ejemplo:

DELETE FROM Autores  
WHERE id\_autor=1;

¿Qué hace?

- Elimina el registro cuyo ID sea 1.

Por eso debe utilizarse con cuidado.

#### Relación con tu proyecto

En el sistema Universo Literario:

#### INSERT

Cuando agregas un libro:

INSERT INTO Libros  
(titulo,id\_autor,id\_editorial,id\_categoria,existencias)  
VALUES (...);

#### SELECT

Cuando buscas un libro:

SELECT \* FROM Libros;

#### UPDATE

Cuando modificas existencias:



UPDATE  
SET  
WHERE id\_libro=5;

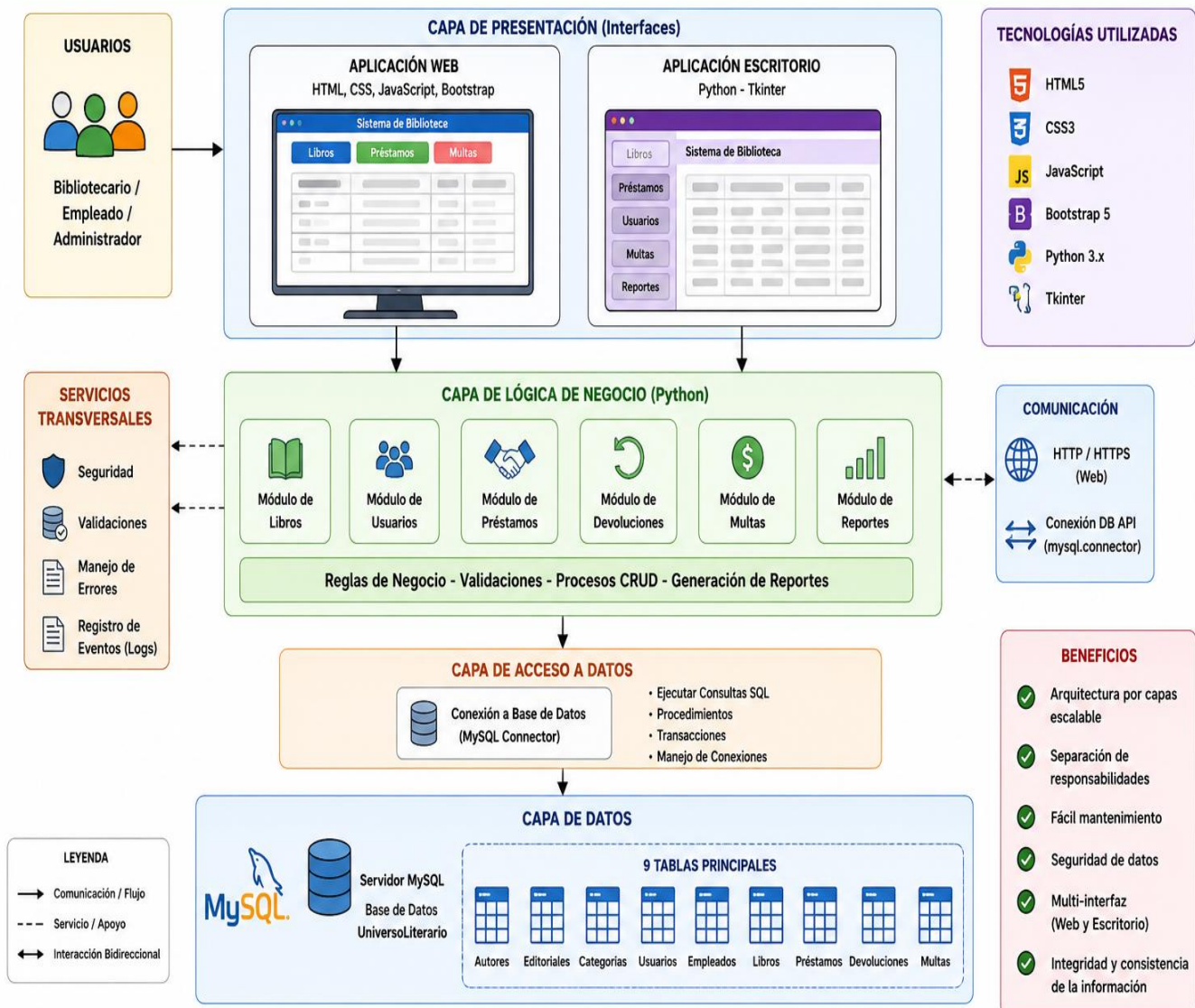
DELETE

Cuando eliminas un libro:

DELETE FROM Libros

Libros  
existencias=10

## DIAGRAMA DE ARQUITECTURA DEL PROYECTO (Sistema de Biblioteca - UniversoLiterario)





Esta imagen representa la arquitectura completa del sistema de biblioteca **"UniversoLiterario"**, mostrando cómo interactúan los usuarios, las aplicaciones, el código en Python y la base de datos MySQL. Es básicamente un mapa que explica el funcionamiento interno del proyecto.

## 1. Usuarios

En el extremo izquierdo aparecen los usuarios del sistema:

Bibliotecario  
Empleado  
Administrador

Estos son quienes utilizan el software para realizar tareas como:

- Registrar libros.
- Registrar usuarios.
- Realizar préstamos.
- Registrar devoluciones.
- Aplicar multas.
- Consultar reportes.

Los usuarios no interactúan directamente con la base de datos; lo hacen mediante una interfaz.

## 2. Capa de Presentación (Interfaces)

Es la parte visual del sistema.

Aquí existen dos formas de usar el sistema:

### Aplicación Web

Tecnologías:

- HTML5
- CSS3
- JavaScript
- Bootstrap 5



Permite acceder desde un navegador web.

Ejemplos:

- Google Chrome
- Edge
- Firefox

El usuario ve formularios, tablas y botones.

Por ejemplo:

[Libros]

[Préstamos]

[ Multas]

Cuando el usuario presiona un botón, la información se envía a la siguiente capa.

### **Aplicación de Escritorio**

Tecnologías:

- Python
- Tkinter

Es un programa instalado en la computadora.

Tiene módulos como:

Libros

Préstamos

Usuarios

Multas

Reportes

Funciona sin necesidad de un navegador.

### **¿Por qué tener dos interfaces?**

Porque el mismo sistema puede utilizarse:

- Desde una página web.
- Desde una aplicación instalada.

Ambas trabajan sobre la misma base de datos.

### **3. Capa de Lógica de Negocio (Python)**

Esta es probablemente la parte más importante.



Aquí se encuentra el código que toma las decisiones.

Cuando un usuario realiza una acción, esta capa decide:

- ✓ Si está permitida.
- ✓ Si los datos son correctos.
- ✓ Qué información guardar.
- ✓ Qué información mostrar.

### **Módulo de Libros**

Gestiona:

- Altas de libros.
- Modificaciones.
- Bajas.
- Consultas.

Ejemplo:

Agregar  
"El Principito"

libro:

La lógica verifica:

- Que el título exista.
- Que la categoría sea válida.
- Que el autor exista.

### **Módulo de Usuarios**

Administra:

- Lectores.
- Empleados.
- Administradores.

Ejemplo:

Registrar usuario

Se valida que no exista previamente.

### **Módulo de Préstamos**



Controla:

- Fecha de préstamo.
- Libro prestado.
- Usuario que lo solicita.

Ejemplo:

Usuario  
Libro 20

15

La lógica verifica:

- Que el libro esté disponible.
- Que el usuario no tenga sanciones.

### **Módulo de Devoluciones**

Controla:

Libro devuelto

Actualiza:

- Estado del libro.
- Fecha de devolución.

### **Módulo de Multas**

Calcula sanciones.

Ejemplo:

3 días de retraso

Puede generar:

\$30 de multa

### **Módulo de Reportes**

Genera estadísticas.

Por ejemplo:

- Libros más prestados.
- Usuarios más activos.
- Multas generadas.
- Préstamos por mes.





## Reglas de Negocio

Debajo de los módulos aparece:

|                        |    |         |
|------------------------|----|---------|
| Reglas                 | de | Negocio |
| Validaciones           |    |         |
| Procesos               |    | CRUD    |
| Generación de Reportes |    |         |

## Validaciones

Comprueban que los datos sean correctos.

Ejemplo:

Nombre vacío

No se permite guardar.

## CRUD

Son las cuatro operaciones básicas de una base de datos:

### Create

Crear registros.

INSERT

### Read

Consultar registros.

SELECT

### Update

Modificar registros.

UPDATE

### Delete

Eliminar registros.

DELETE

## 4. Servicios Transversales

Son funciones que ayudan a todo el sistema.

### Seguridad

Controla:



- Usuarios.
- Contraseñas.
- Permisos.

Por ejemplo:

Un empleado no puede eliminar toda la base de datos.

### **Validaciones**

Revisan que la información tenga sentido.

Ejemplo:

Fecha inválida

### **Manejo de Errores**

Captura errores del sistema.

Ejemplo:

No hay conexión con MySQL

En lugar de cerrar el programa, muestra un mensaje.

### **Registro de Eventos (Logs)**

Guarda actividades importantes.

Ejemplo:

12:30

Usuario admin eliminó libro 25

PM

Esto permite auditorías.

## **5. Comunicación**

A la derecha aparece:

### **HTTP / HTTPS**

Se utiliza para la aplicación web.

Navegador

↓

Servidor

HTTPS agrega cifrado y seguridad.

### **MySQL Connector**



Es la biblioteca que permite a Python comunicarse con MySQL.

Por ejemplo:

```
import mysql.connector
```

Sin ella Python no podría ejecutar consultas SQL.

## **6. Capa de Acceso a Datos**

Esta capa es un intermediario.

Su función es:

- Abrir conexiones.
- Ejecutar consultas SQL.
- Ejecutar procedimientos almacenados.
- Administrar transacciones.

### **Ejemplo**

La lógica de negocio dice:

Registrar préstamo

La capa de acceso traduce eso a SQL:

```
INSERT INTO prestamos (...)
```

y lo envía a MySQL.

## **7. Capa de Datos**

Es donde vive la información permanentemente.

La tecnología utilizada es:

MySQL

Aquí se encuentra la base de datos:

UniversoLiterario

### **Las 9 tablas principales**

#### **Autores**

Guarda:

id\_autor  
nombre



apellido  
nacionalidad

### Editoriales

Guarda información de editoriales.

Planeta  
Penguin

### Categorías

Clasifica los libros.

Novela  
Historia  
Ciencia

### Usuarios

Personas que solicitan libros.

### Empleados

Trabajadores de la biblioteca.

### Libros

Información principal:

Título  
Autor  
Editorial  
Categoría

### Préstamos

Registra:

|       |       |       |    |       |
|-------|-------|-------|----|-------|
| Quién | pidió |       | el | libro |
| Qué   |       | libro |    | pidió |
| Fecha |       |       |    |       |

### Devoluciones

Registra:

|                  |    |         |
|------------------|----|---------|
| Fecha            | de | regreso |
| Estado del libro |    |         |



## Multas

Registra:

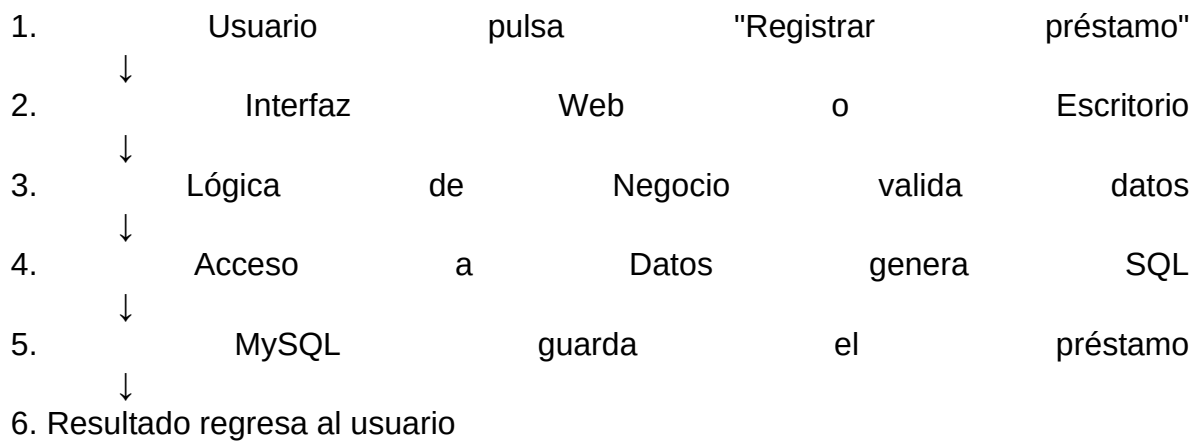
Monto

Motivo

Fecha

## Flujo completo del sistema

Cuando alguien presta un libro ocurre algo así:



## Beneficios de esta arquitectura

La imagen también destaca varias ventajas:

### Arquitectura por capas

Cada parte tiene una función específica.

### Separación de responsabilidades

La interfaz no se mezcla con la base de datos.

### Fácil mantenimiento

Si cambias MySQL o Tkinter, el resto del sistema se ve poco afectado.

### Seguridad de datos



Hay validaciones y control de acceso.

## Multi-interfaz

El mismo sistema funciona:

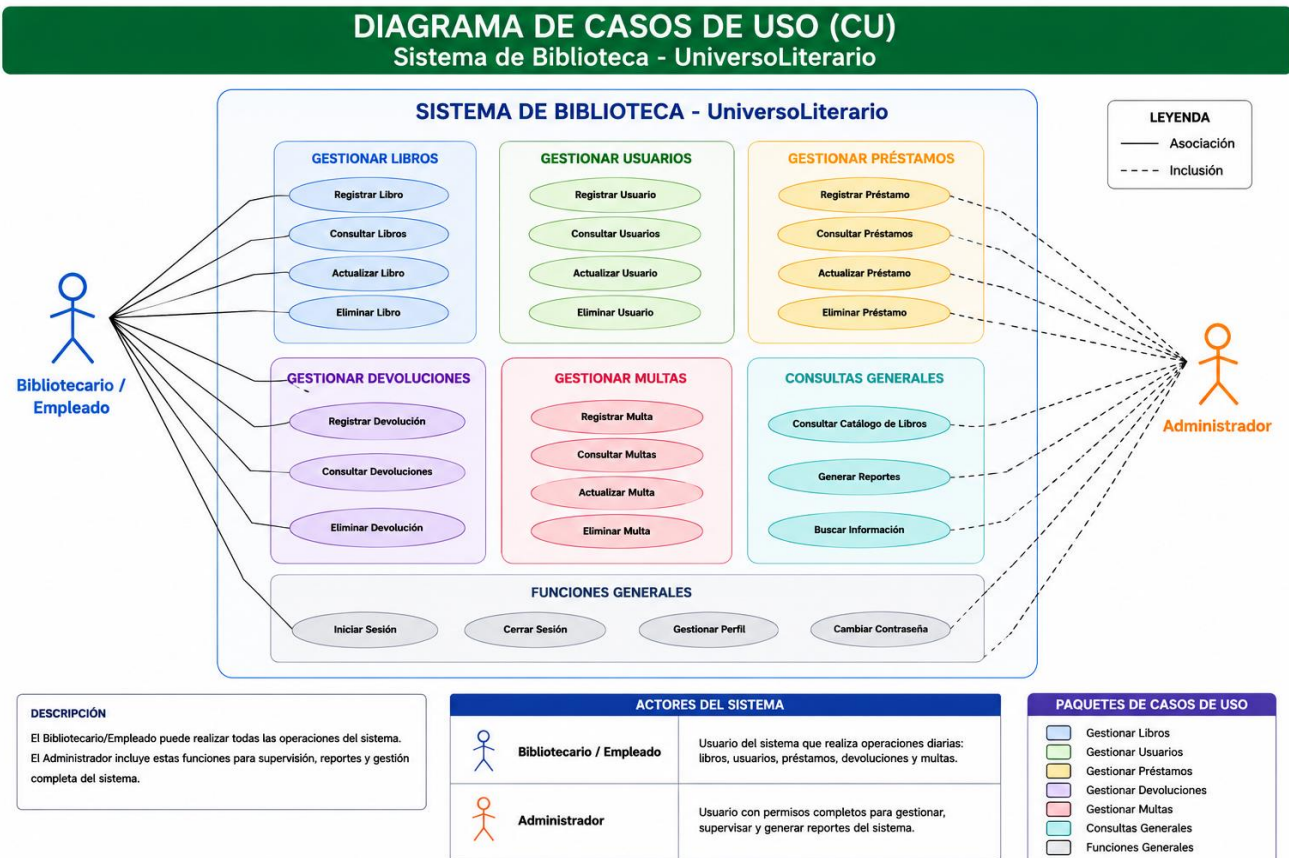
- En navegador web.
- Como aplicación de escritorio.

## Integridad y consistencia

Evita errores como:

- Préstamos duplicados.
- Usuarios inexistentes.
- Libros prestados varias veces al mismo tiempo.

En resumen, la imagen muestra una arquitectura en capas donde los usuarios interactúan con una interfaz (web o escritorio), la lógica de negocio en Python procesa las reglas del sistema, una capa de acceso maneja las consultas SQL y finalmente MySQL almacena toda la información de la biblioteca de forma organizada y segura.





Este Diagrama de Casos de Uso (CU) muestra las funciones principales del sistema de biblioteca **UniversoLiterario** y quiénes pueden utilizarlas.

### **Actores del sistema**

#### **Bibliotecario / Empleado**

Es el usuario que realiza las operaciones diarias de la biblioteca, como:

- Gestionar libros.
- Gestionar usuarios.
- Registrar préstamos.
- Registrar devoluciones.
- Administrar multas.
- Iniciar y cerrar sesión.

#### **Administrador**

Tiene acceso completo al sistema y además puede:

- Supervisar las operaciones.
- Generar reportes.
- Consultar información general.
- Gestionar el sistema de forma global.

### **Casos de uso principales**

#### **Gestionar Libros**

Permite realizar las operaciones CRUD sobre los libros:

- Registrar libro.
- Consultar libros.
- Actualizar libro.
- Eliminar libro.

#### **Gestionar Usuarios**

- Registrar usuario.



- Consultar usuarios.
- Actualizar usuario.
- Eliminar usuario.

### **Gestionar Préstamos**

- Registrar préstamo.
- Consultar préstamos.
- Actualizar préstamo.
- Eliminar préstamo.

### **Gestionar Devoluciones**

- Registrar devolución.
- Consultar devoluciones.
- Eliminar devolución.

### **Gestionar Multas**

- Registrar multa.
- Consultar multas.
- Actualizar multa.
- Eliminar multa.

### **Consultas Generales**

- Consultar catálogo de libros.
- Generar reportes.
- Buscar información.

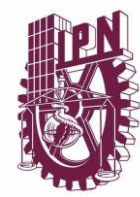
### **Funciones Generales**

Disponibles para los usuarios del sistema:

- Iniciar sesión.
- Cerrar sesión.
- Gestionar perfil.
- Cambiar contraseña.

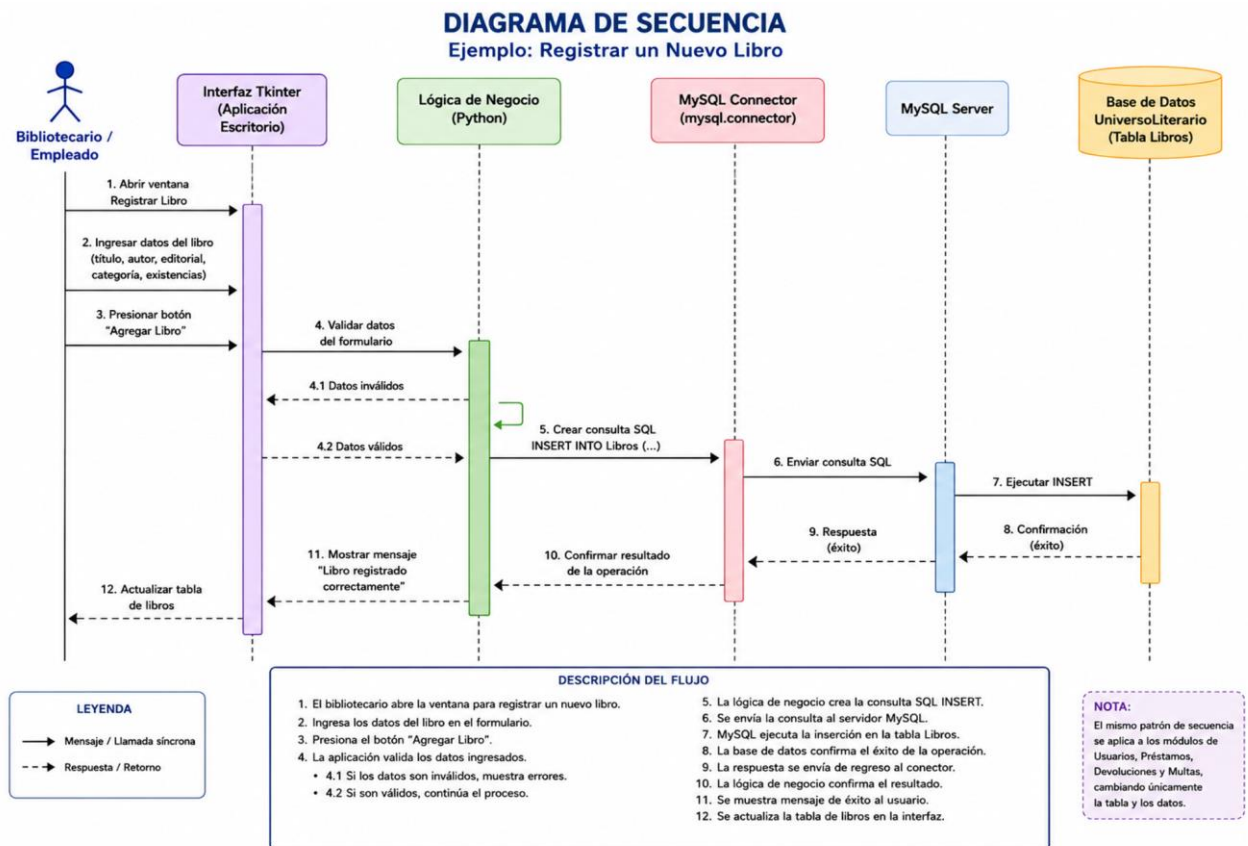
### **Lo más importante del diagrama**





- El sistema está dividido en módulos (Libros, Usuarios, Préstamos, Devoluciones y Multas).
- Tanto el **Bibliotecario/Empleado** como el **Administrador** interactúan con el sistema.
- Cada módulo permite realizar operaciones CRUD (Crear, Consultar, Actualizar y Eliminar).
- El **Administrador** tiene funciones adicionales de supervisión y generación de reportes.
- El objetivo principal del sistema es controlar de forma organizada el manejo de libros, usuarios y préstamos dentro de la biblioteca.

En pocas palabras, este diagrama responde a la pregunta: "**¿Qué puede hacer cada tipo de usuario dentro del sistema?**".



Este Diagrama de Secuencia muestra el proceso que sigue el sistema para registrar un nuevo libro desde que el bibliotecario ingresa los datos hasta que estos quedan guardados en la base de datos.

¿Qué representa?



Mientras que el diagrama de casos de uso responde "qué puede hacer el usuario", el diagrama de secuencia responde:

"¿Cómo interactúan los componentes del sistema paso a paso para realizar una acción?"

Participantes del proceso

Bibliotecario / Empleado

Es quien inicia la operación.

Interfaz Tkinter

La ventana donde el usuario captura los datos del libro.

### **Lógica de Negocio (Python)**

Valida la información y decide qué hacer.

### **MySQL Connector**

Conecta Python con MySQL.

### **MySQL Server**

Ejecuta las consultas SQL.

### **Base de Datos**

Almacena finalmente la información del libro.

## **Flujo principal**

### **1. Abrir formulario**

El bibliotecario abre la ventana **Registrar Libro**.

### **2. Capturar datos**

Ingresa información como:

- Título
- Autor
- Editorial
- Categoría
- Existencias



### 3. Presionar "Agregar Libro"

La interfaz envía los datos al sistema.

### 4. Validación de datos

La lógica de negocio verifica:

- Que no haya campos vacíos.
- Que los datos tengan el formato correcto.
- Que los valores sean válidos.

Si hay errores:

Datos inválidos

El proceso se detiene y se muestra un mensaje al usuario.

### 5. Crear consulta SQL

Si todo es correcto, Python genera una instrucción SQL similar a:

```
INSERT INTO libros (...)  
VALUES (...);
```

### 6. Enviar consulta a MySQL

La consulta pasa por **mysql.connector** y se envía al servidor MySQL.

### 7. Guardar en la base de datos

MySQL ejecuta el comando **INSERT** y almacena el nuevo libro en la tabla correspondiente.

### 8. Confirmación

La base de datos devuelve una respuesta indicando que la operación fue exitosa.

### 9. Retorno del resultado

La confirmación regresa:

|       |    |           |
|-------|----|-----------|
| Base  | de | Datos     |
| ↓     |    |           |
| MySQL |    | Server    |
| ↓     |    |           |
| MySQL |    | Connector |
| ↓     |    |           |



Lógica

de

Negocio



Interfaz

## 10. Mensaje al usuario

Se muestra algo como:

Libro registrado correctamente

## 11. Actualización de la tabla

La interfaz recarga la lista de libros para mostrar el nuevo registro.

### Lo más importante del diagrama

Muestra el flujo completo de comunicación entre el usuario, Python y MySQL.

La **lógica de negocio valida los datos antes de guardarlos**.

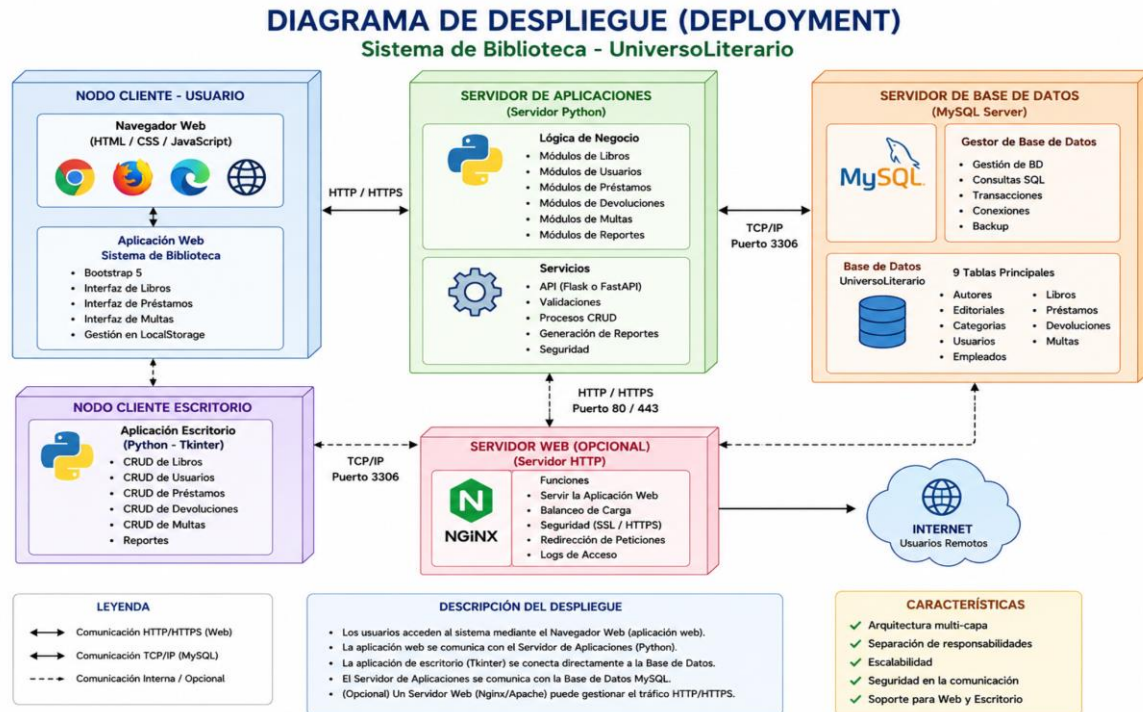
La base de datos nunca recibe información directamente del usuario.

Se utiliza **mysql.connector** como puente entre Python y MySQL.

El comando SQL principal utilizado es:

```
INSERT INTO libros
```

Si los datos son inválidos, el proceso se detiene antes de llegar a la base de datos.



Este Diagrama de Despliegue (Deployment) muestra dónde se ejecuta cada parte del sistema y cómo se comunican entre sí una vez que el proyecto está funcionando.

¿Qué representa?

Mientras que los diagramas anteriores mostraban las funciones y procesos, este diagrama responde:

"¿En qué equipos o servidores se instala el sistema y cómo viaja la información?"

## 1. Nodo Cliente (Usuario)

Es el equipo desde donde trabaja el usuario.

Puede acceder mediante un navegador web como:

- Google Chrome
- Firefox
- Microsoft Edge

Aquí se ejecuta la interfaz web desarrollada con:

- HTML
- CSS
- JavaScript



- Bootstrap

El usuario puede consultar libros, registrar préstamos, gestionar multas, etc.

## **2. Nodo Cliente Escritorio**

También existe una versión de escritorio creada con:

- Python
- Tkinter

Permite realizar:

- CRUD de libros
- CRUD de usuarios
- CRUD de préstamos
- CRUD de devoluciones
- CRUD de multas
- Reportes

Es una alternativa a la versión web.

## **3. Servidor de Aplicaciones (Python)**

Es el "cerebro" del sistema.

Aquí se encuentra:

- Lógica de negocio.
- Validaciones.
- Seguridad.
- Procesos CRUD.
- Generación de reportes.
- API (Flask o FastAPI).

Su función es recibir las solicitudes de los usuarios, procesarlas y comunicarse con la base de datos.

## **4. Servidor de Base de Datos (MySQL)**



Es donde se almacena toda la información.

Contiene la base de datos:

UniversoLiterario

y sus tablas principales:

- Autores
- Editoriales
- Categorías
- Usuarios
- Empleados
- Libros
- Préstamos
- Devoluciones
- Multas

Su trabajo es guardar, consultar y actualizar los datos.

## 5. Servidor Web (Opcional)

En el diagrama aparece Nginx.

Sus funciones son:

- Servir la aplicación web.
- Manejar conexiones HTTP/HTTPS.
- Mejorar la seguridad.
- Gestionar usuarios remotos por Internet.

No es obligatorio para pruebas locales, pero sí recomendable para producción.

## Flujo principal de comunicación

Cuando un usuario registra un libro:

Usuario



Aplicación

Web

o

Escritorio



## Protocolos utilizados

### HTTP / HTTPS

Se utilizan para la comunicación web.

Puertos más comunes:

80 -> HTTP  
443 -> HTTPS

### TCP/IP

Se utiliza para la conexión con MySQL.

Puerto por defecto:

3306

## Lo más importante del diagrama

El sistema está dividido en varias capas (cliente, aplicación y base de datos).

Puede utilizarse tanto desde una aplicación web como desde una aplicación de escritorio.

Python contiene toda la lógica del negocio.

MySQL almacena permanentemente la información.

La comunicación entre componentes se realiza mediante HTTP/HTTPS y TCP/IP.

La arquitectura permite que varios usuarios accedan al sistema al mismo tiempo.





## Tablas principales

### 1. Autores

- Guarda información de los autores.
- Campos: id\_autor, nombre, apellido, nacionalidad.

### 2. Editoriales

- Información de las editoriales.
- Campos: id\_editorial, nombre, telefono.

### 3. Categorías

- Clasificación de los libros.
- Campos: id\_categoria, nombre.

### 4. Libros

- Es la tabla central del sistema.
- Campos: id\_libro, titulo, id\_autor, id\_editorial, id\_categoria, existencias.
- Tiene relaciones con Autores, Editoriales y Categorías.

### 5. Usuarios

- Datos de las personas que solicitan préstamos.



- Campos: id\_usuario, nombre, apellido, telefono.

## 6. Empleados

- Personal encargado de registrar préstamos.
- Campos: id\_empleado, nombre, cargo.

## 7. Préstamos

- Registra cuándo un usuario pide un libro.
- Campos: id\_prestamo, id\_usuario, id\_empleado, fecha\_prestamo.

## 8. Devoluciones

- Controla el regreso de los libros.
- Campos: id\_devolucion, id\_prestamo, fecha\_devolucion.

## 9. Multas

- Registra sanciones por retrasos.
- Campos: id\_multa, id\_usuario, monto, motivo.

---

## Relaciones más importantes

- Un **autor** puede tener muchos **libros**.
- Una **editorial** puede publicar muchos **libros**.
- Una **categoría** puede contener muchos **libros**.
- Un **usuario** puede realizar muchos **préstamos**.
- Un **empleado** puede registrar muchos **préstamos**.
- Un **préstamo** puede tener una **devolución**.
- Un **usuario** puede tener varias **multas**.

## Claves de la base de datos

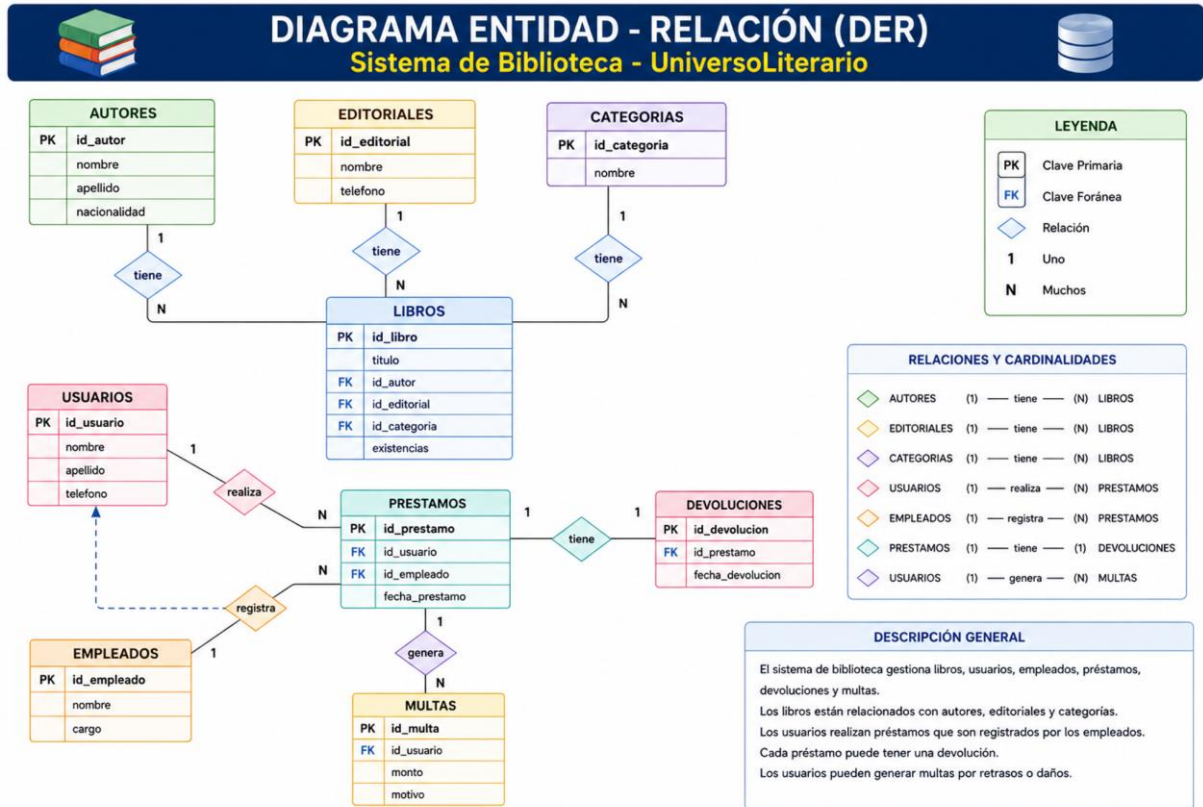
-  **PK (Primary Key)**: Identifica de forma única cada registro.
-  **FK (Foreign Key)**: Relaciona una tabla con otra.

## Idea general

La base de datos está diseñada para **administrar una biblioteca**, permitiendo controlar **libros, autores, editoriales, categorías, usuarios, empleados,**



**préstamos, devoluciones y multas**, manteniendo la integridad de los datos mediante claves primarias y foráneas.



## Diagrama Entidad-Relación (DER)

Este diagrama muestra cómo se relacionan las tablas de la base de datos UniversoLiterario, indicando las entidades, atributos y cardinalidades (1:N, 1:1).

### Entidades principales

#### LIBROS (Entidad central)

Contiene la información de los libros:

- id\_libro (PK)
- titulo
- id\_autor (FK)
- id\_editorial (FK)
- id\_categoria (FK)
- existencias



Es la tabla más importante porque se conecta con:

- Autores
- Editoriales
- Categorías

## AUTORES

Almacena:

- id\_autor
- nombre
- apellido
- nacionalidad

**Relación:** Un autor puede tener muchos libros (**1:N**).

## EDITORIALES

Almacena:

- id\_editorial
- nombre
- telefono

**Relación:** Una editorial puede publicar muchos libros (**1:N**).

## CATEGORÍAS

Almacena:

- id\_categoria
- nombre

**Relación:** Una categoría puede contener muchos libros (**1:N**).

## USUARIOS

Almacena:



- id\_usuario
- nombre
- apellido
- telefono

#### **Relaciones:**

- Un usuario puede realizar muchos préstamos (**1:N**).
- Un usuario puede generar muchas multas (**1:N**).

### **EMPLEADOS**

Almacena:

- id\_empleado
- nombre
- cargo

**Relación:** Un empleado puede registrar muchos préstamos (**1:N**).

### **PRÉSTAMOS**

Almacena:

- id\_prestamo
- id\_usuario (FK)
- id\_empleado (FK)
- fecha\_prestamo

Es la tabla que registra cuándo un usuario solicita un libro.

### **DEVOLUCIONES**

Almacena:

- id\_devolucion
- id\_prestamo (FK)
- fecha\_devolucion



**Relación:** Cada préstamo tiene una devolución (**1:1**).

## MULTAS

Almacena:

- id\_multa
- id\_usuario (FK)
- monto
- motivo

Registra penalizaciones por retrasos o daños.

## Relaciones más importantes

| Relación              | Cardinalidad |
|-----------------------|--------------|
| Autor → Libros        | 1 : N        |
| Editorial → Libros    | 1 : N        |
| Categoría → Libros    | 1 : N        |
| Usuario → Préstamos   | 1 : N        |
| Empleado → Préstamos  | 1 : N        |
| Préstamo → Devolución | 1 : 1        |
| Usuario → Multas      | 1 : N        |





### ¿QUÉ ES?

HTTPS (HyperText Transfer Protocol Secure) es la versión segura de HTTP. Cifra la información que viaja entre tu navegador y el servidor.



El candado indica que la conexión es segura y el certificado es válido.



### ¿POR QUÉ ES IMPORTANTE?

- Protege datos sensibles (contraseñas, pagos, formularios).
- Evita que terceros intercepten la información.
- Genera confianza en los usuarios.
- Es un factor clave para el SEO.

### HTTP vs HTTPS

#### HTTP (NO SEGURO)

La información viaja en texto claro.



CUALQUIERA PUEDE INTERCEPTAR LOS DATOS

#### HTTPS (SEGURO)

La información viaja cifrada.



LOS DATOS ESTÁN PROTEGIDOS

**HTTPS (HyperText Transfer Protocol Secure)** es la versión segura de HTTP. Su función principal es **cifrar la información** que se transmite entre el navegador y el servidor para que nadie pueda leerla o modificarla.

#### Puntos clave

- **Protege datos sensibles** como:
  - Contraseñas
  - Información bancaria
  - Formularios
  - Datos personales
- **Evita la interceptación de información** por parte de terceros.
- El **candado** que aparece junto a la URL indica que la conexión es segura y que el certificado del sitio es válido.

#### Diferencia entre HTTP y HTTPS

##### HTTP

No es seguro

Los datos viajan en texto plano

La información puede ser interceptada

##### HTTPS

Es seguro

Los datos viajan cifrados

La información está protegida





## HTTP

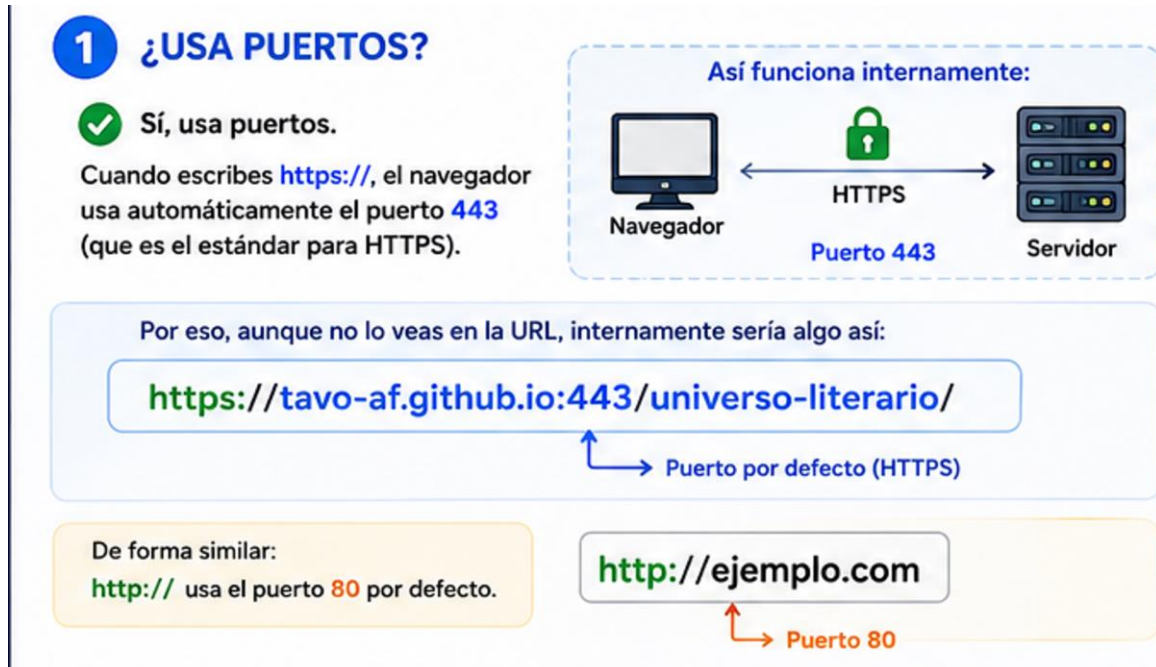
No muestra candado de seguridad

## HTTPS

Muestra candado de seguridad

### Conclusión breve

HTTPS es un protocolo de seguridad esencial para cualquier sitio web, ya que protege la información de los usuarios mediante cifrado, evita accesos no autorizados a los datos y genera mayor confianza durante la navegación y las transacciones en línea.



**HTTPS sí utiliza puertos**, aunque normalmente no aparecen visibles en la dirección web.

### Puerto de HTTPS

- Cuando escribes una dirección que comienza con **https://**, el navegador utiliza automáticamente el **puerto 443**.
- El puerto 443 es el **puerto estándar de HTTPS** y permite que la comunicación entre el navegador y el servidor viaje de forma segura y cifrada.

### Ejemplo interno:

**https://tavo-af.github.io:443/universo-literario/**

Aunque el puerto 443 no se muestre en la URL, el navegador lo utiliza por defecto.

### Funcionamiento

La comunicación ocurre entre:





- Navegador
- Servidor

Utilizando:

- Protocolo HTTPS
- Puerto 443
- Cifrado de datos

### Comparación con HTTP

- HTTP utiliza el **puerto 80** por defecto.
- HTTPS utiliza el **puerto 443** por defecto.

### Protocolo Puerto por defecto

HTTP 80

HTTPS 443



- HTTPS significa conexión segura y cifrada.
- Protege la información que viaja por Internet.
- Utiliza el puerto 443 por defecto.



## 2. Usuario de GitHub: **tavo-af**

Es el nombre de usuario de GitHub del propietario del sitio.

- Identifica quién publicó la página.
- En este caso, el sitio pertenece al usuario **tavo-af**.

## 3. Dominio de GitHub Pages: **github.io**

Es el dominio que GitHub proporciona gratuitamente para alojar páginas web.

- Permite publicar sitios web directamente desde un repositorio.
- No es necesario comprar un dominio propio para usarlo.

## 4. Repositorio o Proyecto: **universo-literario**

Es el nombre del repositorio donde están almacenados los archivos de la página web.

- Contiene el código HTML, CSS, JavaScript e imágenes.
- GitHub Pages toma los archivos de este repositorio para mostrar el sitio.

## ¿Cómo funciona?

1. El usuario escribe la URL en el navegador.
2. La solicitud viaja por Internet mediante HTTPS.
3. GitHub Pages recibe la solicitud.
4. GitHub busca el archivo principal (**index.html**).
5. Se carga y muestra la página web al usuario.

## Estructura visual de la URL

| Parte    | Significado       |
|----------|-------------------|
| https:// | Protocolo seguro  |
| tavo-af  | Usuario de GitHub |



| Parte              | Significado             |
|--------------------|-------------------------|
| github.io          | Dominio de GitHub Pages |
| universo-literario | Repositorio o proyecto  |

## ETAPA 2

### ¿Qué es Oracle?

Oracle es un Sistema Gestor de Bases de Datos Relacional (RDBMS) desarrollado por Oracle Corporation. Su principal función consiste en almacenar, administrar y proteger grandes cantidades de información, permitiendo que múltiples usuarios accedan a ella de forma simultánea sin comprometer la integridad de los datos.

Una de sus principales características es la confiabilidad que ofrece para aplicaciones empresariales, ya que incorpora mecanismos avanzados de seguridad, recuperación ante fallos, administración de usuarios y optimización de consultas. Gracias a ello, Oracle es utilizado por instituciones financieras, empresas de telecomunicaciones, gobiernos y organizaciones que requieren manejar información crítica.

Además, Oracle soporta procedimientos almacenados, funciones, disparadores (triggers), vistas, índices y herramientas de administración que facilitan el desarrollo de aplicaciones de gran escala.

### Características de Oracle

Oracle destaca por ofrecer una amplia variedad de funcionalidades orientadas al entorno empresarial. Entre las más importantes se encuentran:

- Alta disponibilidad de los datos.
- Seguridad mediante autenticación y control de privilegios.
- Escalabilidad para soportar millones de registros.
- Recuperación automática ante fallos.
- Optimización de consultas mediante índices.
- Compatibilidad con diferentes plataformas y lenguajes de programación.
- Administración eficiente de transacciones.

Estas características permiten desarrollar aplicaciones confiables incluso cuando cientos o miles de usuarios trabajan al mismo tiempo.



## ¿Qué es Solace?

Solace es una plataforma de mensajería basada en eventos que permite la comunicación entre aplicaciones de forma rápida, segura y confiable. Su función principal consiste en actuar como intermediario entre diferentes sistemas, enviando mensajes sin que las aplicaciones tengan que comunicarse directamente entre sí.

A diferencia de una comunicación tradicional donde un sistema espera la respuesta de otro, Solace utiliza un modelo de mensajería asíncrona, permitiendo que los mensajes sean enviados y recibidos cuando cada aplicación esté disponible.

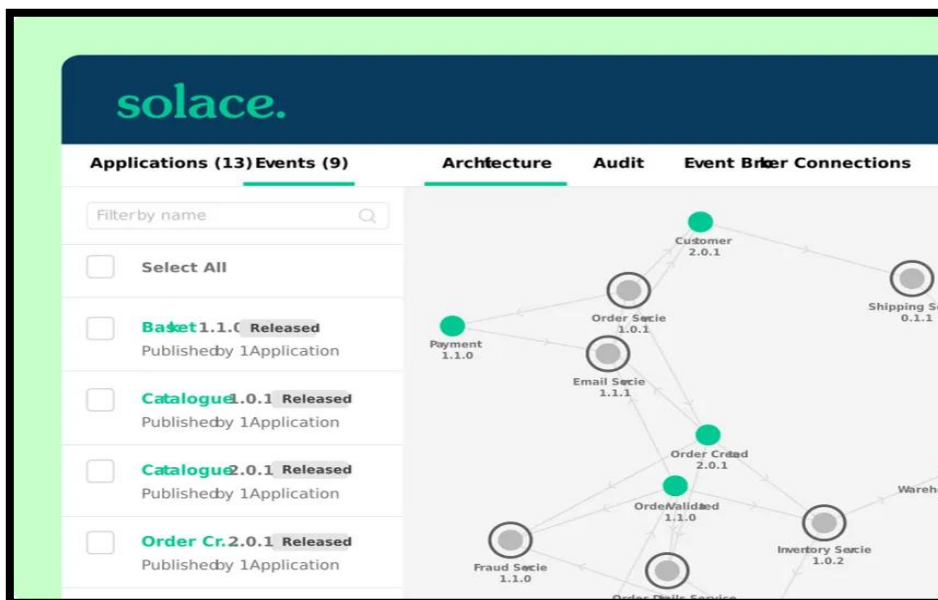
Esto facilita la integración de sistemas desarrollados en distintos lenguajes de programación, servidores o incluso ubicados en diferentes redes.

## Historia de Solace

Solace Corporation fue fundada en el año 2001 con el objetivo de ofrecer soluciones de mensajería para aplicaciones empresariales de alto rendimiento.

Con el paso del tiempo, la empresa evolucionó desde dispositivos físicos especializados (hardware appliances) hacia plataformas completamente virtuales y servicios en la nube, permitiendo que empresas de cualquier tamaño puedan implementar arquitecturas orientadas a eventos.

Actualmente, Solace es ampliamente utilizado en sectores como banca, telecomunicaciones, salud, manufactura y comercio electrónico, donde la velocidad de transmisión de datos es un aspecto fundamental.





La arquitectura del proyecto está organizada por capas para facilitar el desarrollo y mantenimiento del sistema.

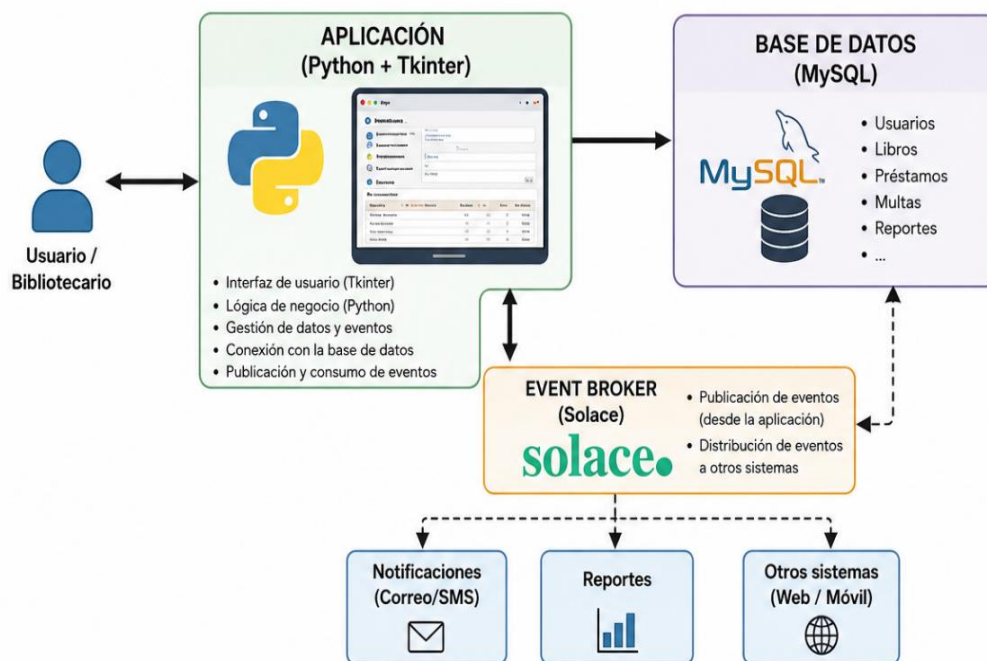
En primer lugar se encuentra la capa de presentación, donde interactúan los usuarios mediante la aplicación. Posteriormente, las solicitudes son procesadas por la capa de lógica de negocio, encargada de validar la información y ejecutar las reglas del sistema.

Después interviene la capa de acceso a datos, responsable de establecer la comunicación con Oracle para realizar operaciones de consulta, inserción, actualización o eliminación de registros.

Como complemento, se incorpora Solace, el cual funciona como una capa de mensajería orientada a eventos. Su función consiste en distribuir información entre diferentes módulos del sistema mediante eventos y mensajes asíncronos, permitiendo una comunicación rápida, desacoplada y escalable.

Finalmente, Oracle almacena toda la información de manera permanente y segura.

## FUNCIONAMIENTO GENERAL DEL SISTEMA (UNIFICADO)





## Funcionamiento del sistema:

El sistema está compuesto por una **única aplicación desarrollada en Python**, la cual integra la **interfaz gráfica (Tkinter)** y la **lógica de negocio**. Esta aplicación se comunica con una **base de datos MySQL** para almacenar y consultar información, y con un **Event Broker (Solace)** para publicar eventos que serán consumidos por otros servicios, como notificaciones, reportes o aplicaciones externas.

### 1. Inicio de la aplicación

Cuando el usuario ejecuta el sistema, se inicia la aplicación desarrollada en **Python**. En ella se carga la interfaz gráfica creada con **Tkinter**, se establece la conexión con la base de datos **MySQL** y se configura la conexión con **Solace** para el envío de eventos.

### 2. Interacción del usuario:

El usuario (bibliotecario o administrador) realiza sus operaciones desde la interfaz gráfica, por ejemplo:

- Registrar usuarios.
- Registrar libros.
- Realizar préstamos y devoluciones.
- Buscar información.
- Consultar reportes.
- Actualizar o eliminar registros.

Cada acción realizada en la interfaz genera una solicitud que es procesada por el código de Python.

### 3. Procesamiento de la lógica de negocio:

Python recibe la acción realizada por el usuario y ejecuta la lógica correspondiente.

Durante este proceso el sistema:

- Valida que los datos ingresados sean correctos.
- Verifica reglas de negocio (por ejemplo, que un libro esté disponible para préstamo).
- Calcula multas si existen retrasos.
- Controla el flujo completo de cada operación.
- Determina qué información debe almacenarse o actualizarse.





Toda esta lógica se encuentra implementada mediante funciones, clases o módulos dentro del código.

#### 4. Comunicación con la base de datos (MySQL):

Después de validar la información, Python se conecta a **MySQL** mediante un conector (como mysql-connector-python o PyMySQL) para ejecutar operaciones **CRUD**:

- **Create:** insertar nuevos registros.
- **Read:** consultar información.
- **Update:** modificar registros existentes.
- **Delete:** eliminar registros.

Por ejemplo, cuando se registra un préstamo, el sistema:

1. Consulta si el libro está disponible.
2. Inserta el préstamo en la base de datos.
3. Cambia el estado del libro a "Prestado".
4. Guarda la fecha de devolución.

Toda esta información queda almacenada permanentemente en MySQL.

#### 5. Publicación de eventos mediante Solace:

Una vez que la operación fue realizada correctamente, la aplicación publica un evento en **Solace**.

Estos eventos representan acciones importantes del sistema, por ejemplo:

- Usuario registrado.
- Libro agregado.
- Préstamo realizado.
- Devolución registrada.
- Libro vencido.
- Multa generada.

El objetivo es que otros sistemas puedan reaccionar automáticamente sin afectar el funcionamiento de la aplicación principal.

#### 6. Consumo de eventos por otros sistemas:

Los eventos enviados a Solace pueden ser consumidos por diferentes servicios, como:



- **Sistema de notificaciones**, que envía correos electrónicos o mensajes SMS.
- **Sistema de reportes**, que actualiza estadísticas o genera informes.
- **Aplicaciones web o móviles**, que muestran la información en tiempo real.

Gracias al Event Broker, estos sistemas funcionan de manera independiente de la aplicación principal.

## 7. Actualización de la interfaz:

Después de completar la operación, Python devuelve el resultado a la interfaz gráfica.

La interfaz:

- Muestra mensajes de éxito o error.
- Actualiza las tablas con la información más reciente.
- Refresca los datos mostrados al usuario.
- Permite continuar realizando nuevas operaciones.

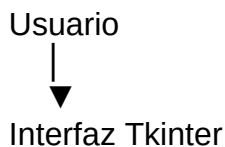
Todo este proceso ocurre de manera transparente para el usuario.

## Flujo completo del sistema:

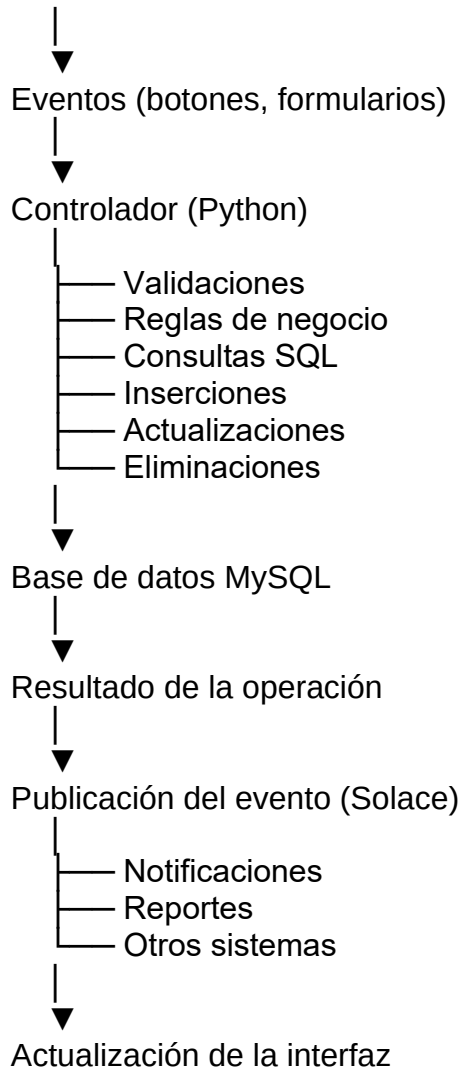
1. El usuario realiza una acción desde la interfaz gráfica (Tkinter).
2. Python recibe la solicitud.
3. Se validan los datos y las reglas del negocio.
4. Python ejecuta operaciones en la base de datos MySQL.
5. Se confirma que la operación fue exitosa.
6. Python publica un evento en Solace.
7. Los sistemas externos reciben el evento y ejecutan sus procesos (notificaciones, reportes, aplicaciones web o móviles).
8. Python actualiza la interfaz mostrando el resultado al usuario.

## Funcionamiento dentro del código:

Dentro del proyecto, el flujo del código normalmente sigue esta estructura:







En esta arquitectura, Tkinter forma parte de la misma aplicación desarrollada en Python, por lo que la interfaz y la lógica de negocio trabajan de manera integrada. Python actúa como el núcleo del sistema: controla la interfaz, procesa las operaciones, gestiona la comunicación con MySQL y publica eventos en Solace para mantener sincronizados los demás servicios.

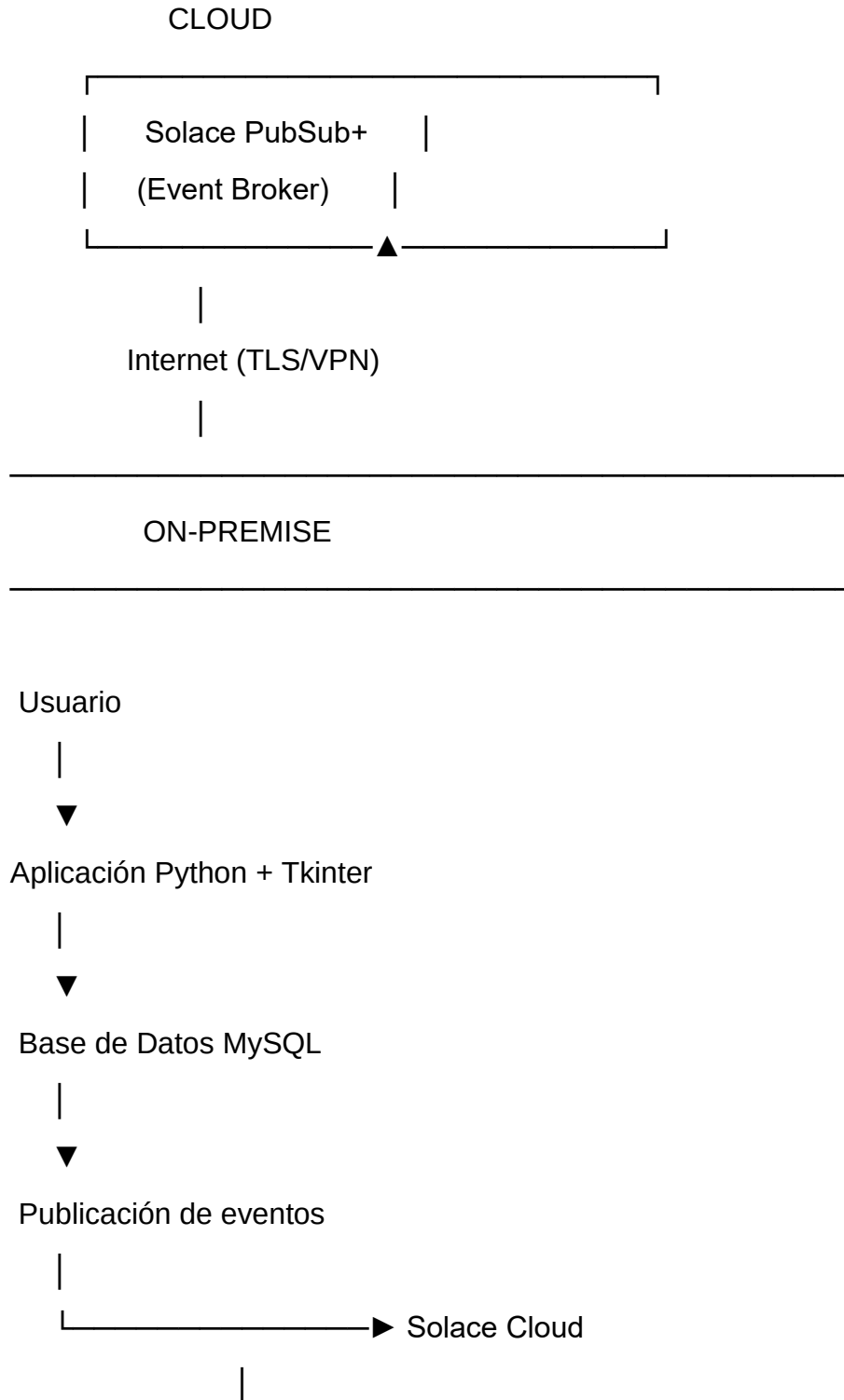
Nuestra arquitectura utiliza Python + Tkinter + MySQL + Solace, la conexión On-Premise Cloud (también llamada arquitectura híbrida) significa que parte de la aplicación se encuentra dentro de la infraestructura de la organización (on-premise) y otra parte utiliza servicios alojados en la nube.

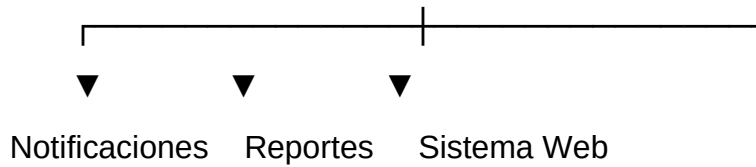
¿Qué es una conexión On-Premise Cloud?

Una conexión On-Premise Cloud es la comunicación segura entre los sistemas instalados localmente (en una empresa, universidad o biblioteca) y los servicios que se encuentran en la nube.



En nuestro caso, la arquitectura funciona de la siguiente manera:





## Funcionamiento de la conexión:

### 1. Aplicación local (On-Premise):

La aplicación desarrollada en Python con Tkinter se instala en las computadoras de la biblioteca.

Desde esta aplicación los usuarios pueden:

- Registrar libros.
- Registrar usuarios.
- Realizar préstamos.
- Registrar devoluciones.
- Consultar reportes.

Toda la lógica de negocio se ejecuta localmente.

### 2. Base de datos local:

La aplicación se conecta a un servidor MySQL que también se encuentra dentro de la organización.

Por ejemplo:

Python

|



mysql.connector.connect()

|



Servidor MySQL

Aquí se almacenan:

- Usuarios.



- Libros.
- Préstamos.
- Multas.
- Historial.

La información permanece bajo el control de la institución.

### 3. Conexión hacia la nube:

Cuando ocurre un evento importante, la aplicación no envía directamente correos o reportes.

En su lugar publica un mensaje en Solace.

Por ejemplo:

Libro prestado

Python crea un mensaje:

```
{  
  "evento": "prestamo",  
  "usuario": 25,  
  "libro": 120,  
  "fecha": "2026-07-09"  
}
```

Ese mensaje viaja por Internet utilizando una conexión segura.

Python

|

TLS (HTTPS/TCP)

|

Internet

|

Solace Cloud

### 4. Solace en la nube

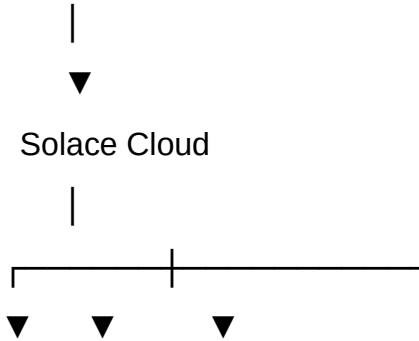


Solace recibe el evento y lo distribuye a todos los sistemas que estén suscritos.

Por ejemplo:

Evento:

Prestamo realizado



Correo Reporte Web

Así la aplicación principal no necesita conocer quién utilizará la información.

## 5. Sistemas consumidores:

Otros sistemas reciben el evento automáticamente.

Por ejemplo:

Sistema de correo

Envía un correo al usuario indicando la fecha de devolución.

Sistema de reportes

Actualiza las estadísticas de préstamos.

Aplicación web

Actualiza el catálogo en tiempo real.

Ventajas de una arquitectura híbrida (On-Premise + Cloud)

- Mayor seguridad: La información crítica permanece en los servidores de la organización.



- Escalabilidad: Los servicios en la nube pueden crecer según la demanda sin afectar la aplicación local.
- Alta disponibilidad: Si se agregan nuevos servicios (como una app móvil), basta con suscribirlos a los eventos publicados en Solace.
- Desacoplamiento: La aplicación principal no depende directamente de los sistemas de notificaciones, reportes o aplicaciones externas.
- Integración sencilla: Es posible conectar nuevos sistemas sin modificar la lógica principal de la aplicación.

Ejemplo aplicado a una biblioteca:

Supongamos que un bibliotecario registra un préstamo.

1. El bibliotecario realiza el préstamo desde la aplicación de escritorio en Tkinter.
2. Python valida que el usuario exista y que el libro esté disponible.
3. La información se guarda en MySQL, ubicado en los servidores de la biblioteca.
4. Una vez confirmado el préstamo, Python publica el evento "Préstamo realizado" en Solace Cloud mediante una conexión segura (TLS).
5. Solace distribuye el evento a los sistemas suscritos.
6. El sistema de notificaciones envía un correo al usuario con la fecha de devolución.
7. El sistema de reportes actualiza las estadísticas de préstamos.
8. Si existe una aplicación web o móvil, esta recibe el evento y refleja inmediatamente el cambio en el estado del libro.

En este modelo, la aplicación y la base de datos permanecen On-Premise, mientras que Solace funciona como un puente en la nube para integrar y sincronizar los distintos servicios. Esta combinación ofrece un equilibrio entre el control de los datos locales y las ventajas de la computación en la nube, como la escalabilidad, la integración y la comunicación en tiempo real.



## MySQL (Base de Datos)

En esta arquitectura, MySQL funciona como el sistema encargado de almacenar toda la información del proyecto. Cada vez que el usuario realiza una acción desde la interfaz, como registrar un libro, agregar un usuario, realizar un préstamo o consultar información, la aplicación desarrollada en Python envía las solicitudes a MySQL para guardar, modificar o recuperar los datos correspondientes.

Además de almacenar la información de manera permanente, MySQL garantiza la integridad y consistencia de los datos mediante tablas relacionadas, llaves primarias y foráneas, restricciones y transacciones. Esto permite que el sistema pueda manejar de forma organizada los registros de libros, usuarios, préstamos, devoluciones y multas, evitando pérdidas de información y facilitando consultas rápidas cuando el bibliotecario necesita acceder a los datos.

## Solace (Event Broker)

En este proyecto, **Solace** actúa como un **Event Broker**, es decir, como un intermediario encargado de distribuir eventos y mensajes generados por la aplicación. En lugar de que la aplicación tenga que comunicarse directamente con todos los módulos del sistema, únicamente envía el evento a Solace, quien se encarga de reenviarlo a los servicios correspondientes.

Por ejemplo, cuando un bibliotecario registra un préstamo desde la aplicación en Python, la información se almacena primero en MySQL y, posteriormente, la aplicación publica un evento en Solace. Este evento puede ser utilizado por distintos módulos para realizar acciones adicionales, como enviar una notificación por correo electrónico o SMS al usuario, generar un reporte de préstamos realizados o compartir la información con otros servicios web sin afectar el funcionamiento de la aplicación principal.

Gracias a este mecanismo, el sistema mantiene una arquitectura desacoplada, donde cada componente puede trabajar de manera independiente. Esto facilita la incorporación de nuevas funcionalidades en el futuro, ya que basta con conectar un nuevo servicio a Solace para que reciba los eventos generados por la aplicación, sin necesidad de modificar el código existente.



## Diagrama de flujo: Proceso de un préstamo

El proceso inicia cuando el bibliotecario selecciona al usuario y el libro que desea prestar mediante la interfaz desarrollada en Tkinter. La información es enviada al programa en Python, el cual se encarga de procesar la solicitud y verificar los datos ingresados.

Posteriormente, Python consulta la base de datos MySQL para comprobar que el libro se encuentre disponible. Si la validación es correcta, el sistema registra el préstamo y actualiza el inventario para reflejar la nueva disponibilidad del ejemplar.

Finalmente, Solace genera un evento asociado al préstamo realizado, permitiendo la ejecución de acciones adicionales como notificaciones o reportes. Una vez completado el proceso, la interfaz muestra un mensaje de confirmación indicando que el préstamo fue registrado exitosamente.

### DIAGRAMA DE FLUJO: PROCESO DE UN PRÉSTAMO



## Diagrama de arquitectura del sistema

El sistema está organizado mediante una arquitectura por capas, lo que permite separar las funciones de cada componente y facilita el mantenimiento, la escalabilidad y el desarrollo del proyecto. Cada capa cumple una función específica y se comunica con la siguiente para procesar las solicitudes del usuario de forma ordenada.

La primera capa corresponde a la capa de presentación, donde se encuentra la interfaz desarrollada con Tkinter. Es el punto de contacto entre el usuario y el sistema, ya que desde aquí el bibliotecario puede administrar usuarios, libros, préstamos, reportes y notificaciones. La interfaz únicamente recopila la información y la envía a la aplicación, sin realizar procesos complejos.

En la capa de lógica de negocio, desarrollada en Python, se ejecutan todas las reglas del sistema. Esta capa recibe las solicitudes provenientes de la interfaz, valida la información ingresada y determina las acciones que deben realizarse, además de coordinar la comunicación con la base de datos y generar eventos cuando es necesario.



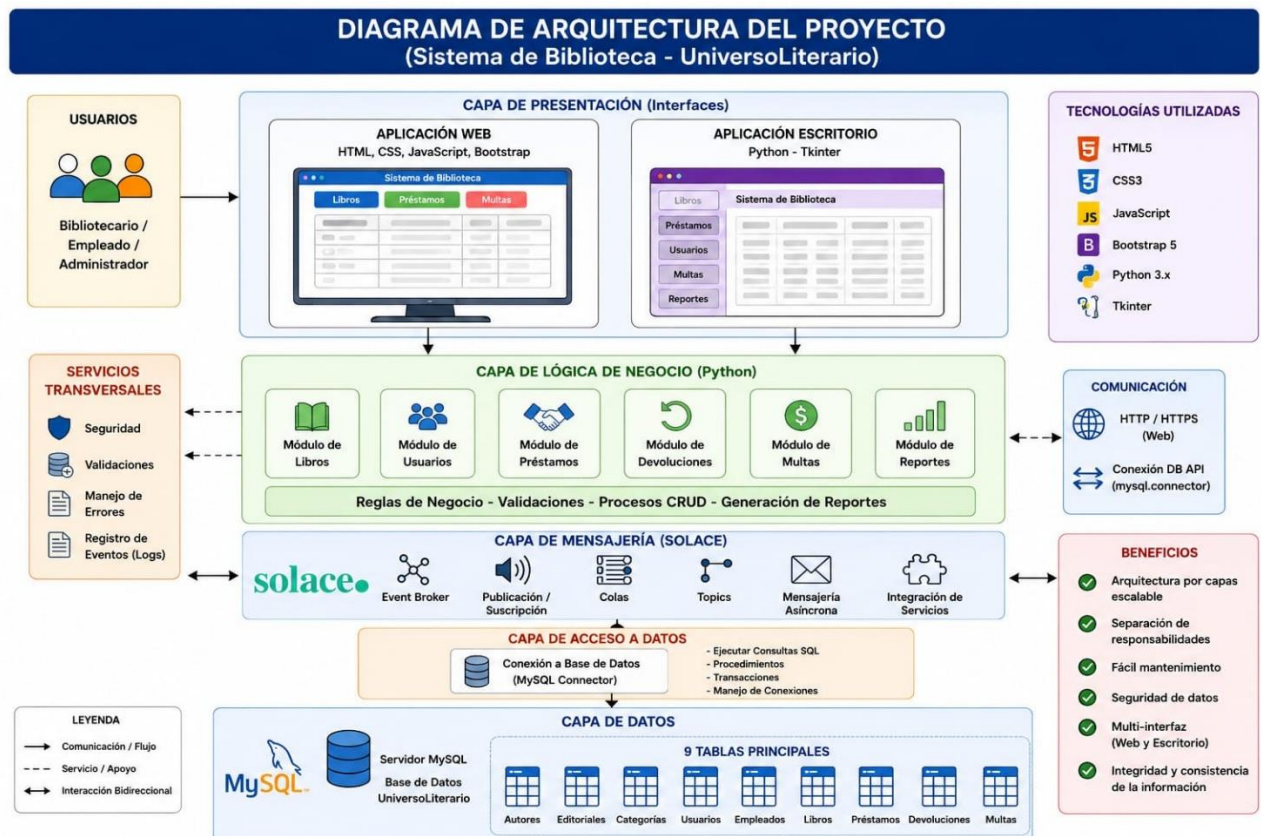


Como parte de la arquitectura se incorpora una capa de mensajería, donde se encuentra Solace actuando como Event Broker. Su función es recibir los eventos publicados por la aplicación y distribuirlos a los servicios que los necesiten, como el envío de notificaciones, la generación de reportes, estadísticas o la comunicación con otros sistemas mediante APIs.

Gracias a ello, la aplicación principal no necesita ejecutar todas estas tareas directamente, haciendo que el sistema sea más flexible y escalable.

Después se encuentra la capa de acceso a datos, la cual utiliza el conector de Python con MySQL para ejecutar consultas SQL, insertar, actualizar, eliminar y recuperar información de la base de datos. Esta capa funciona como intermediaria entre la aplicación y el gestor de base de datos.

Finalmente, la capa de datos está conformada por MySQL, donde se almacena toda la información del sistema, incluyendo usuarios, libros, préstamos y demás registros. Además de guardar los datos de forma permanente, MySQL garantiza la integridad, seguridad y disponibilidad de la información para que el sistema funcione correctamente.





## Conclusión:

El desarrollo de la base de datos **Universo Literario** en MySQL permitió diseñar e implementar un sistema capaz de administrar de manera organizada y eficiente los recursos de una biblioteca. A través de las tablas relacionadas de autores, editoriales, categorías, libros, usuarios, empleados, préstamos, devoluciones y multas, se logró mantener la integridad y el control de la información mediante el uso de claves primarias y foráneas.

Durante el proyecto se aplicaron los conceptos fundamentales de MySQL y del lenguaje SQL, utilizando sentencias **DDL** para la creación de la estructura de la base de datos y **DML** para la manipulación de los registros. Además, se comprendió la importancia del diseño de bases de datos, las relaciones entre tablas y la gestión eficiente de la información.

En conclusión, **Universo Literario** representa una solución funcional para la administración de bibliotecas, ya que facilita el control de libros, usuarios y préstamos, reduce errores en el manejo de datos y demuestra la utilidad de MySQL como herramienta para el desarrollo de sistemas de información reales.

Oracle y Solace representan dos tecnologías que cumplen funciones distintas pero complementarias dentro de una arquitectura moderna de software. Oracle garantiza un almacenamiento seguro, confiable y eficiente de la información, mientras que Solace facilita la comunicación entre aplicaciones mediante eventos y mensajería asíncrona.

La integración de ambas herramientas permite desarrollar sistemas más escalables, flexibles y preparados para responder a grandes volúmenes de información y múltiples usuarios. Además, la arquitectura por capas utilizada en el proyecto favorece una mejor organización del código, simplifica el mantenimiento y facilita la incorporación de nuevas funcionalidades en el futuro.